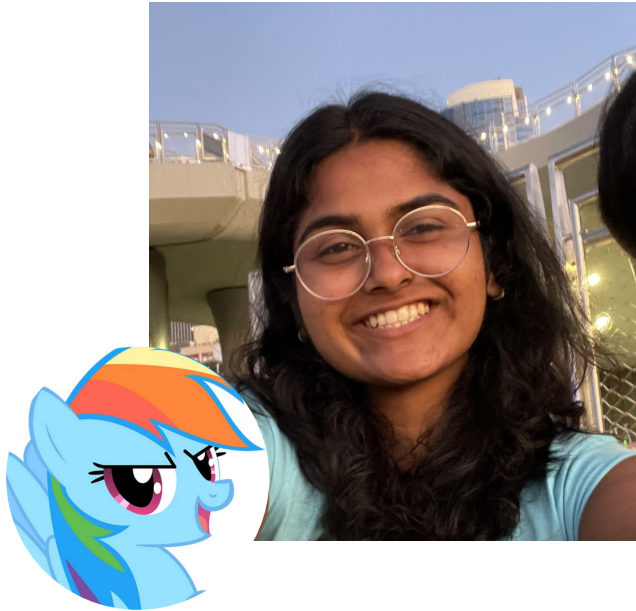


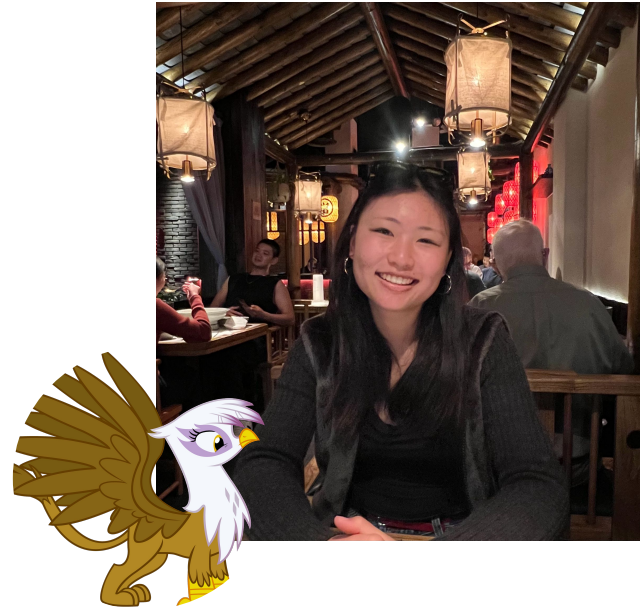
CS 61B TRS 3

4/3/26

Introductions



Esha Potharaju
3rd year, CS + Ethnic
Studies Major



Amanda Zhang
2nd year, EECS Major

Contents

1. Heaps
 - a. Q1
 - b. Q2
2. Graph Algorithms (Traversals, Shortest paths, MST)
 - a. Q3
 - b. Q4
3. Hashing
 - a. Q7
 - b. Q5
 - c. Q6

Heaps

CS 61B MT2 Review Session



Why Priority Queues?

Consider an *unusually* crowded Hospital Emergency Room.

Patients arrive at different times, with different assigned levels of urgency.

Using a PQ, we can access all current cases in order of priority = urgency.

* Even if a patient with a minor issue arrived earlier, a more urgent case (higher priority) will be treated first.

Consider an infinitely large amount of patients !
Luckily, our PQ tells us who's "up next" in **constant** time.
(No need to search!)



Priority Queues.

Priority Queue (ADT) = immediately get item of highest priority

```
/** (Min) Priority Queue: Allowing tracking and removal of  
 * the smallest item in a priority queue. */  
public interface MinPQ<Item> {  
    public void add(Item x);  
    public Item getMin();  
    public Item removeMin();  
    public int size();  
}
```

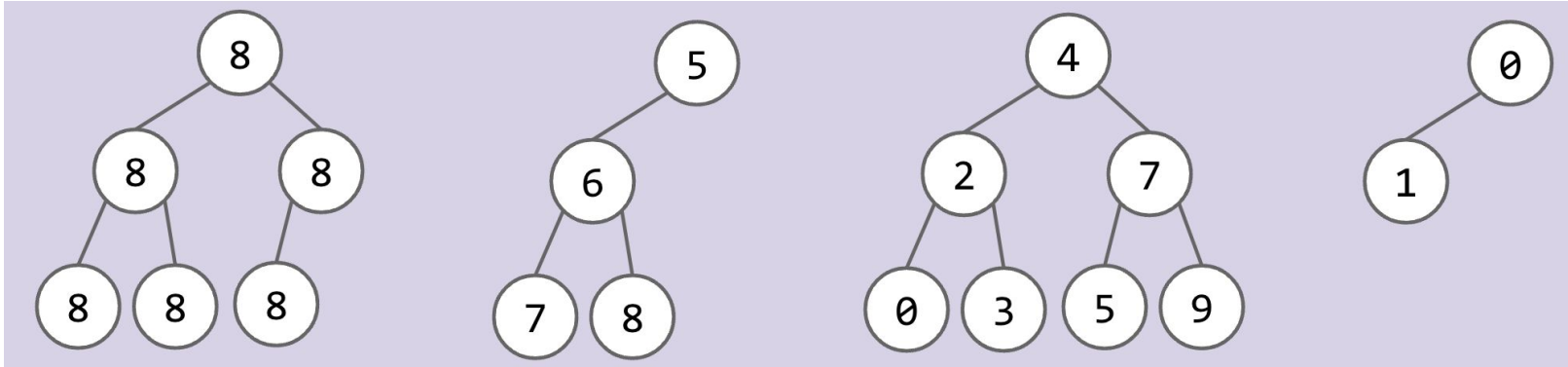
ADT = multiple implementations = many possible runtimes for each operation

- ordered array
- BST
- HashTable
- ...



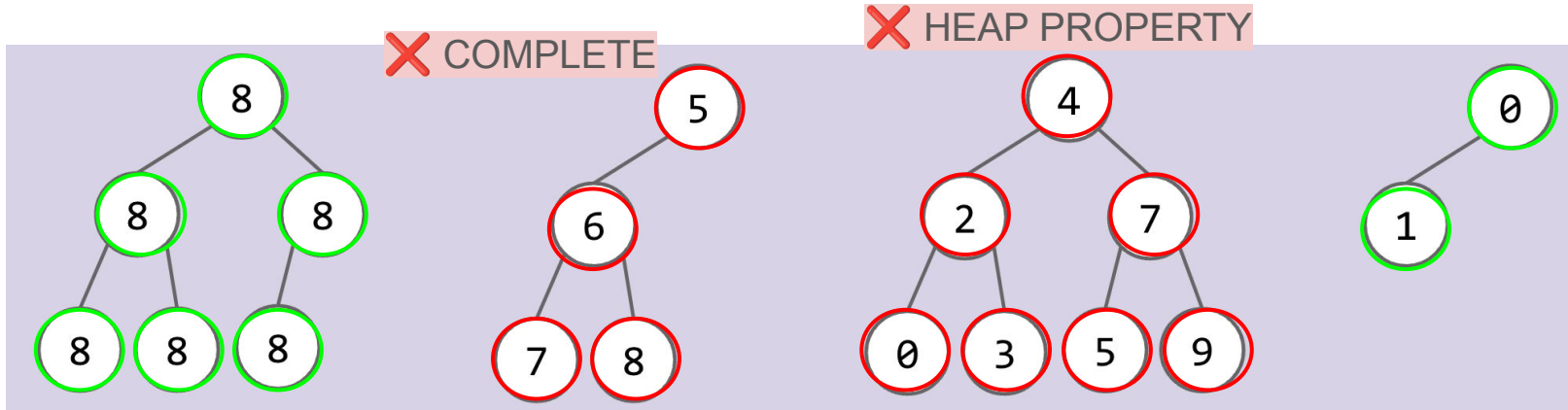
What is a Heap? (Formal Definition)

- A heap is a special kind of binary tree that obeys 2 key properties:
 1. **Heap Property:**
 - **Min-heap:** every node is less than or equal to its children
 - **Max-heap:** every node is greater than or equal to its children
 2. **Complete:** missing items only at bottom level (if any), all nodes as far left as possible
- Which of the following binary trees is a heap?



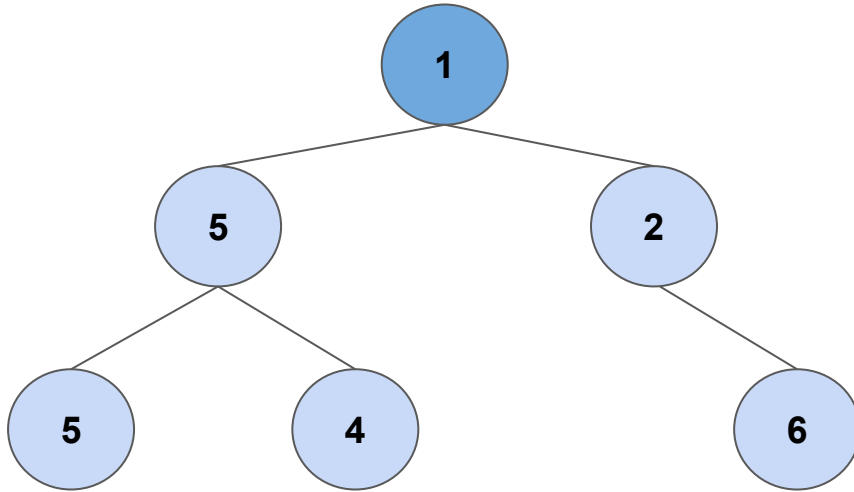
What is a Heap? (Formal Definition)

- A heap is a special kind of binary tree that obeys 2 key properties:
 1. **Heap Property:**
 - **Min-heap:** every node is less than or equal to its children
 - **Max-heap:** every node is greater than or equal to its children
 2. **Complete:** missing items only at bottom level (if any), all nodes as far left as possible
- Which of the following binary trees is a heap?



Checkpoint!



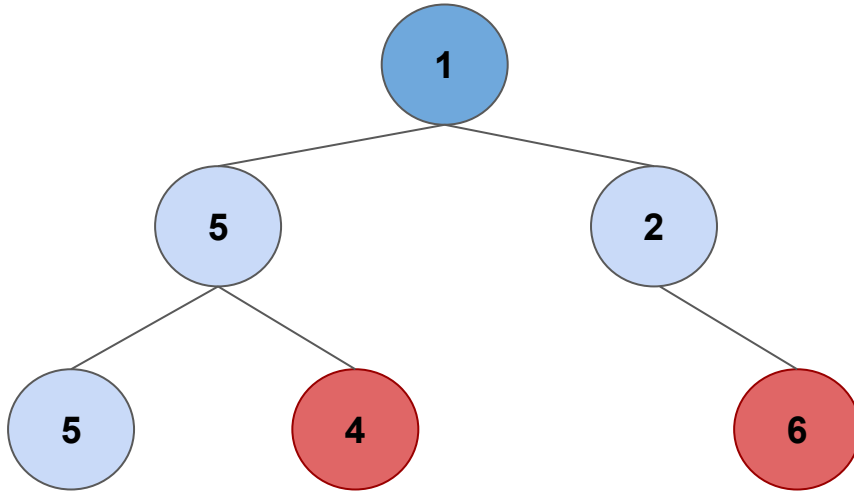


Checkpoint! ⚠

Before anything else, does this look like a valid Min-Heap?

- ☐ Min-Heap Property?
- ☐ Completeness Property?





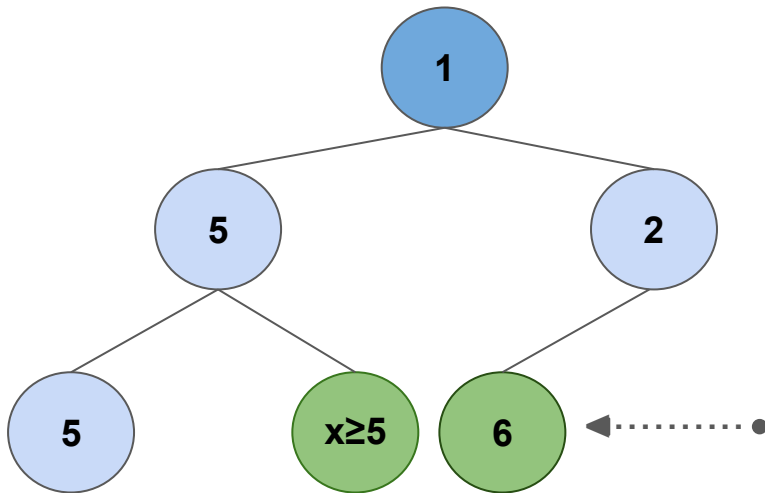
Checkpoint! ⚠

Before anything else, does this look like a valid Min-Heap?

~~☐ Min-Heap Property?~~

~~☐ Completeness Property?~~





Checkpoint! ⚠

Before anything else, does this look like a valid Min-Heap?

☐ **Min-Heap Property** ✓

☐ **Completeness Property** ✓

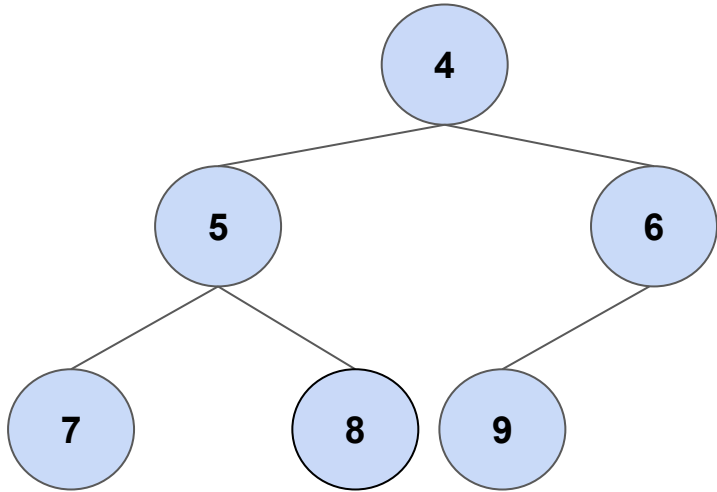
We can now keep going!



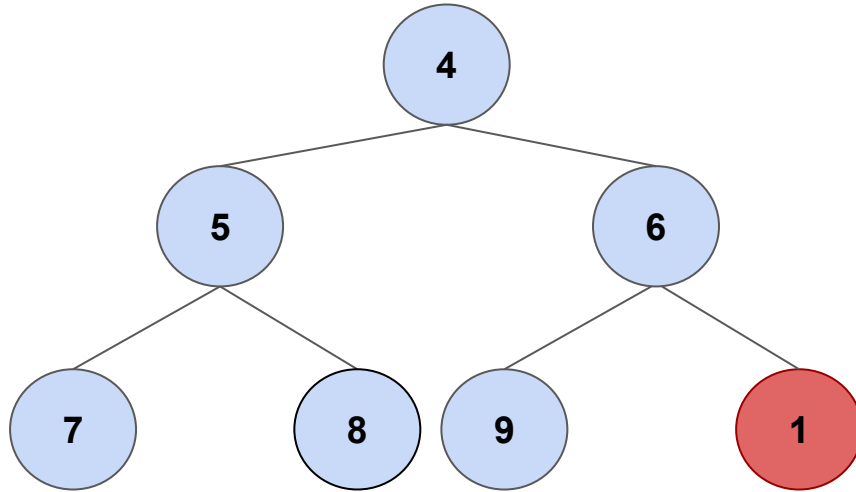
Let's insert and remove values in our Min-Heap.



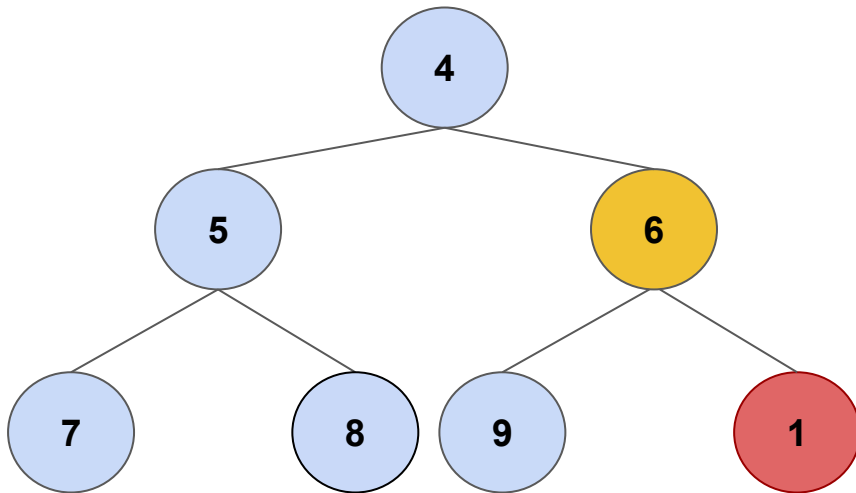
Let's add new element! `add(1)`



Add to the end!



Bubble Up!



What is Bubble Up? ⚠

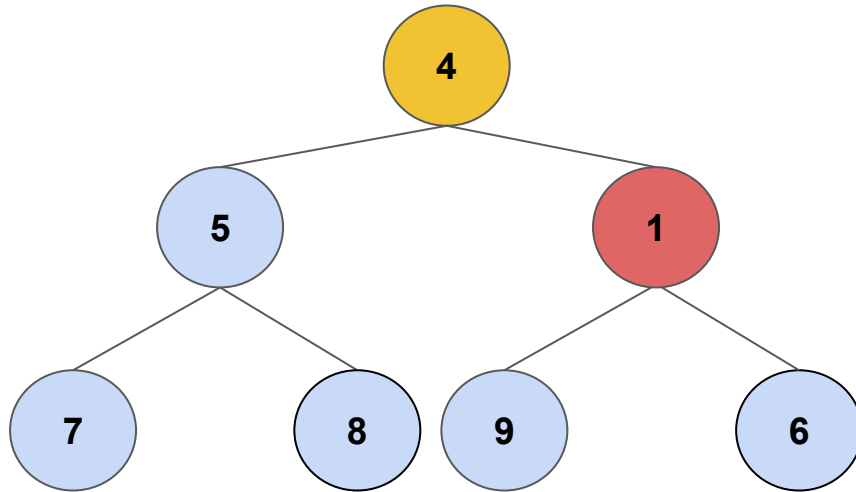
If the element that has just been inserted is **N**, swap **N** with its parent as long as **N** is smaller than its parent or until **N** is the new root.

If **N** is equal to its parent, you can either swap the items or not.

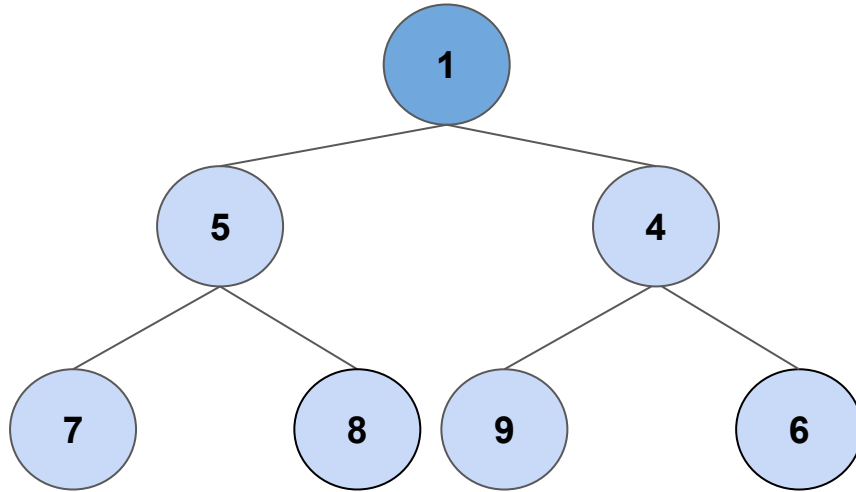
☐ Min-Heap Property ✓



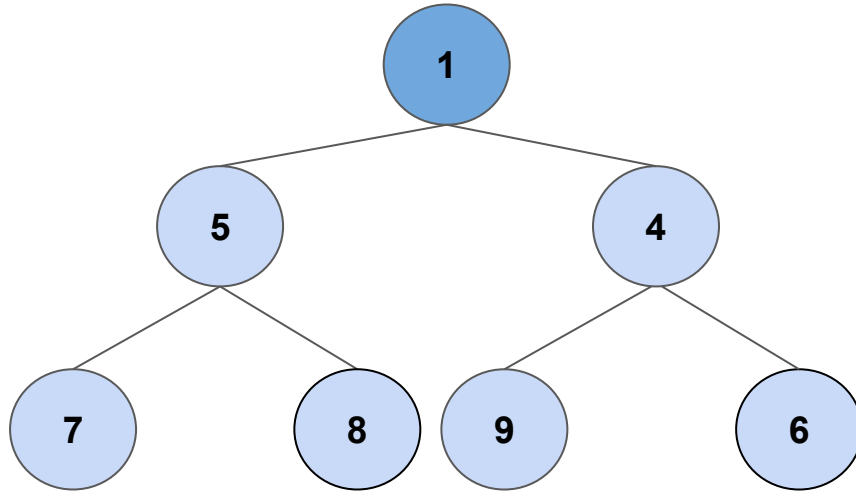
Bubble Up!



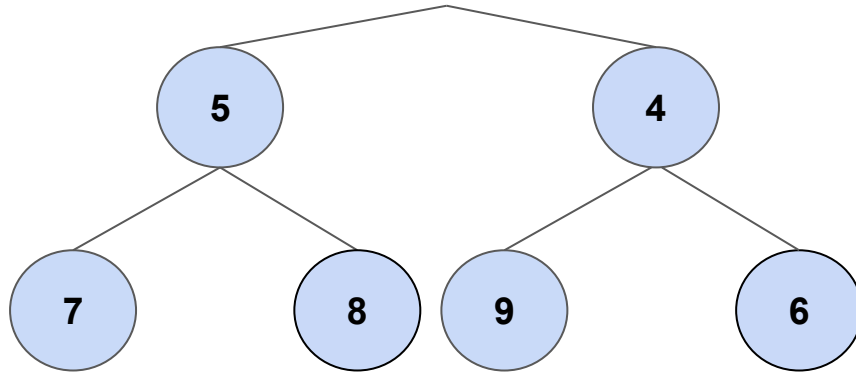
Done!



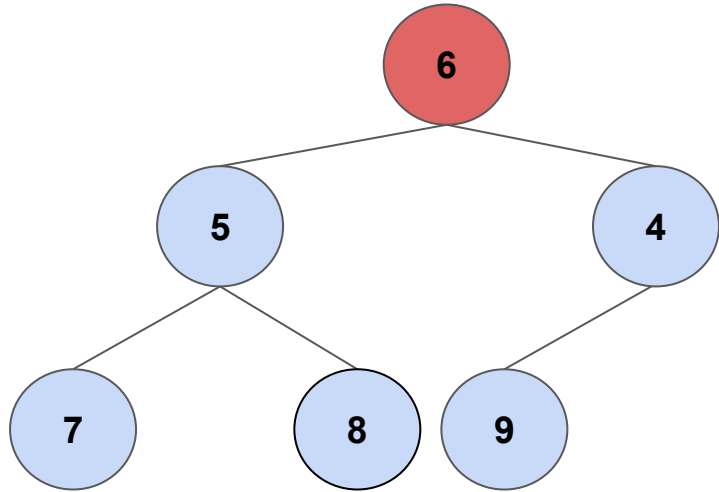
Let's remove smallest element! removeMin()



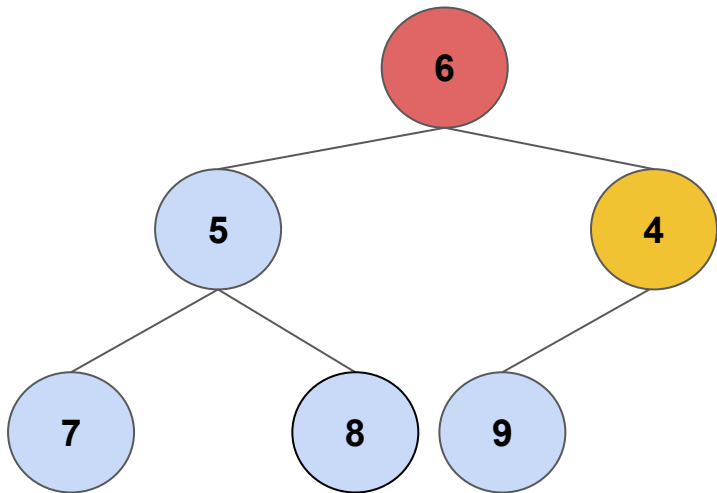
Remove root!



Promote last element!



Bubble Down!



What is Bubble Down? ⚠

If the new root **N** is greater than either of its children, swap it with that child.

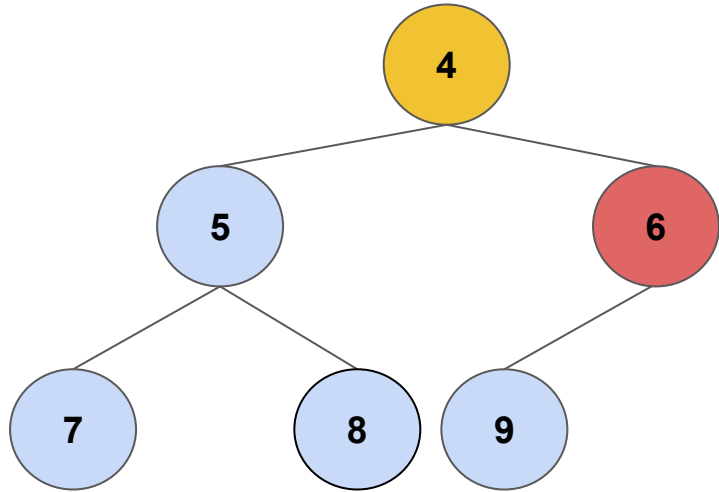
If it is greater than both of its children, choose the smaller of the two children.

Continue swapping **N** with its children in the same manner until **N** is smaller than its children or it has no children.

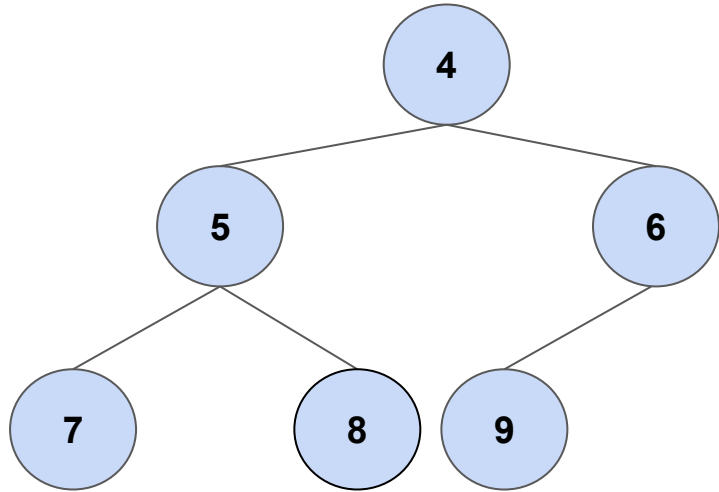
☑ **Min-Heap Property** ✓



Bubble Down!



Done!



Why Heaps? Runtime Analysis!

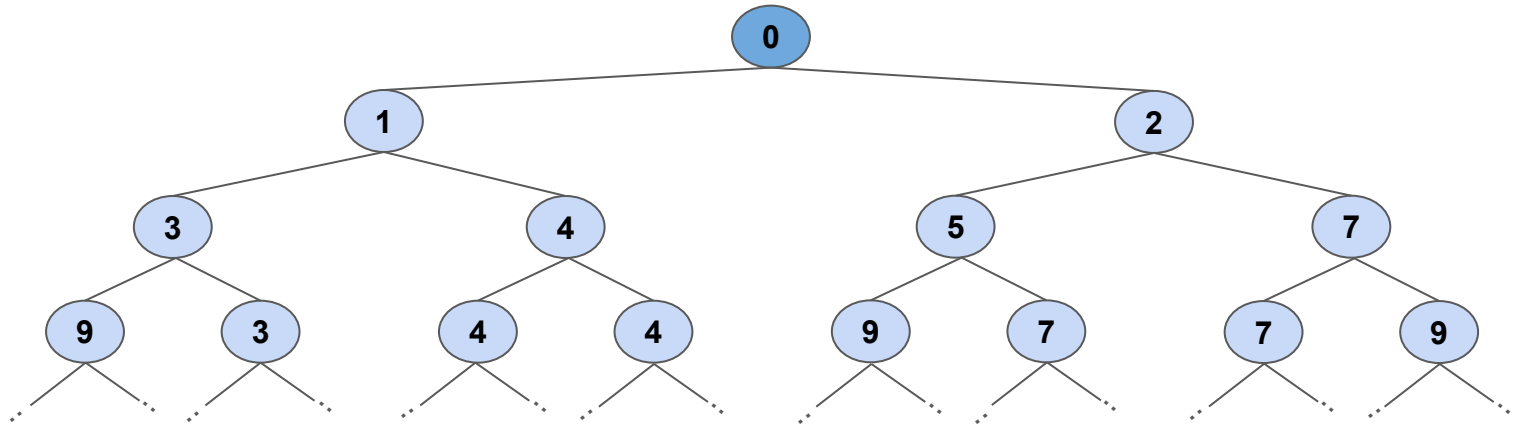
Data structures we've seen so far...

	<code>.add(x)</code>	<code>.getMin(x)</code>	<code>.removeMin(x)</code>
Ordered Array	$\Theta(N)$	$\Theta(1)$	$\Theta(N)$
Hash Table	$\Theta(1)$	$\Theta(N)$	$\Theta(N)$
Balanced BST	$\Theta(\log N)$	$\Theta(\log N)$	$\Theta(\log N)$
Min Heap (PQ)	$\Theta(\log N)$	$\Theta(1)$	$\Theta(\log N)$

Worst Case $\Theta(\cdot)$ Runtimes



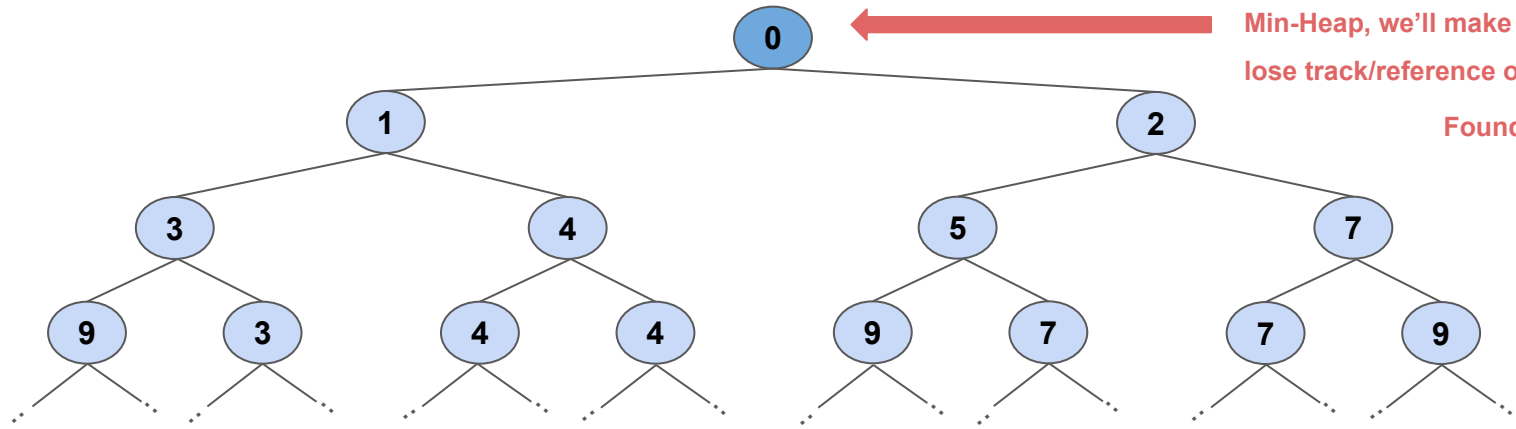
“Think Big”



An **infinitely** 🐝 large amount of intermediate layers later...



“Think Big”



However we implement our
Min-Heap, we'll make it so we never
lose track/reference of our root!

Found in $\Theta(1)$!

An **infinitely** 🐝 large amount of intermediate layers later...



Questions



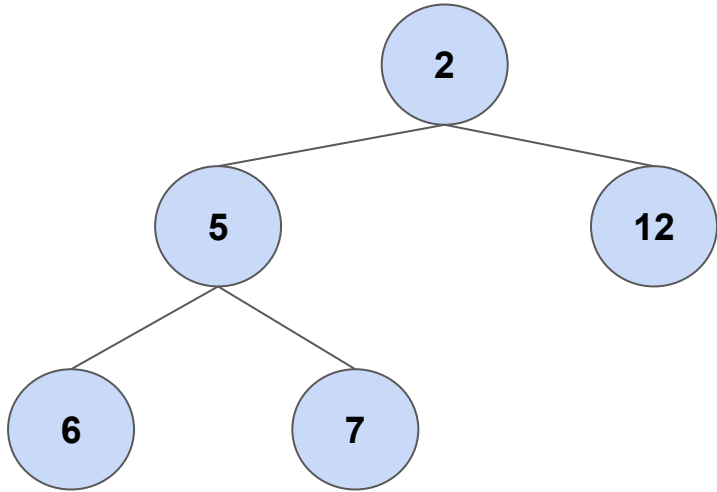
1 Heaperations



1a



1a

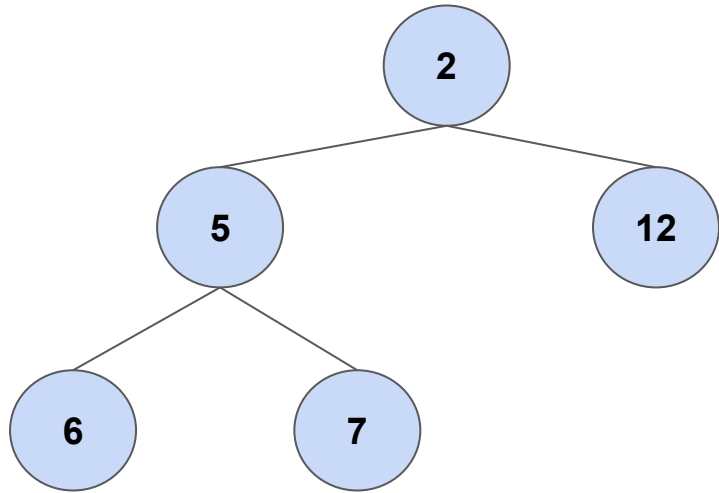


Consider this min-heap.

Draw the heap after
inserting 4, 13, 3 in that
order.



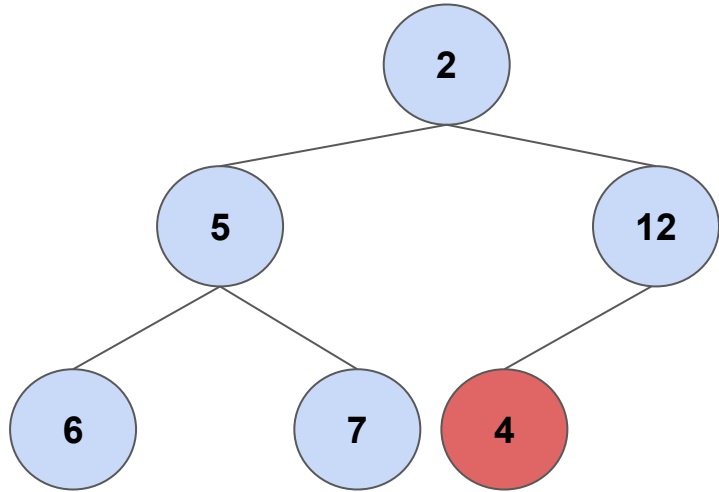
1a



add(4)



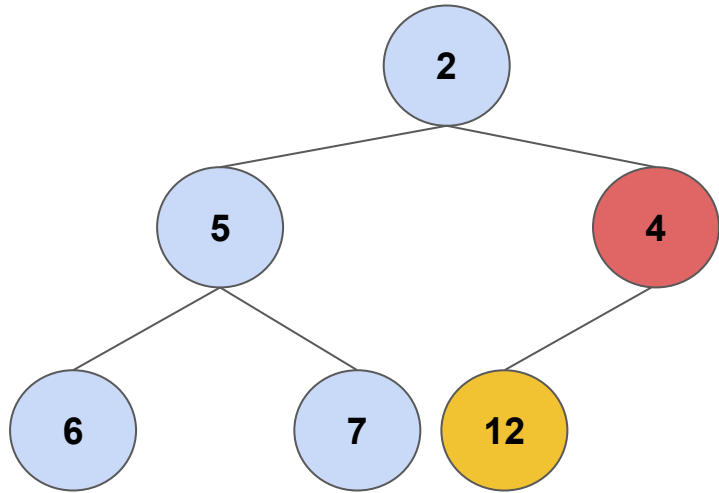
1a



Add to the end



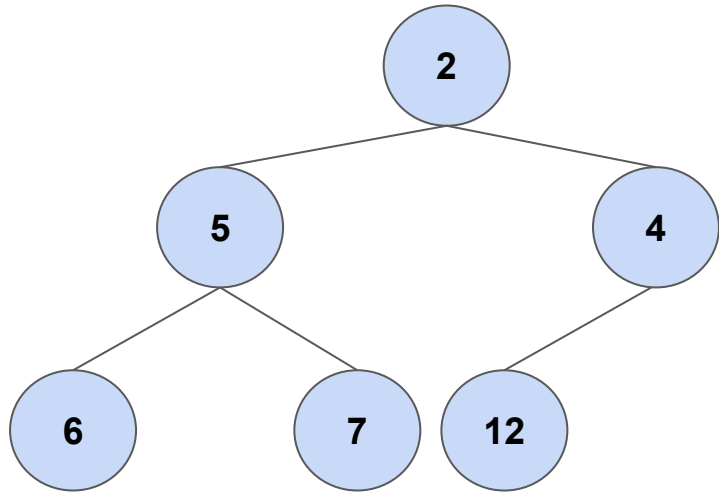
1a



Bubble Up



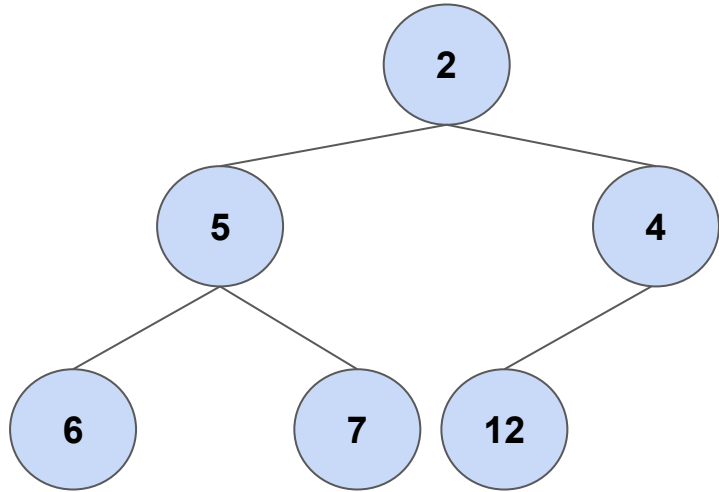
1a



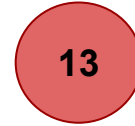
Done!



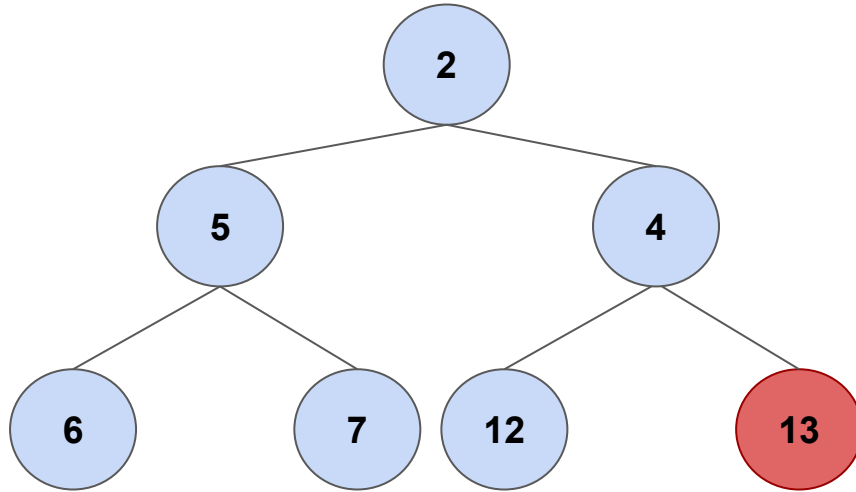
1a



add(13)



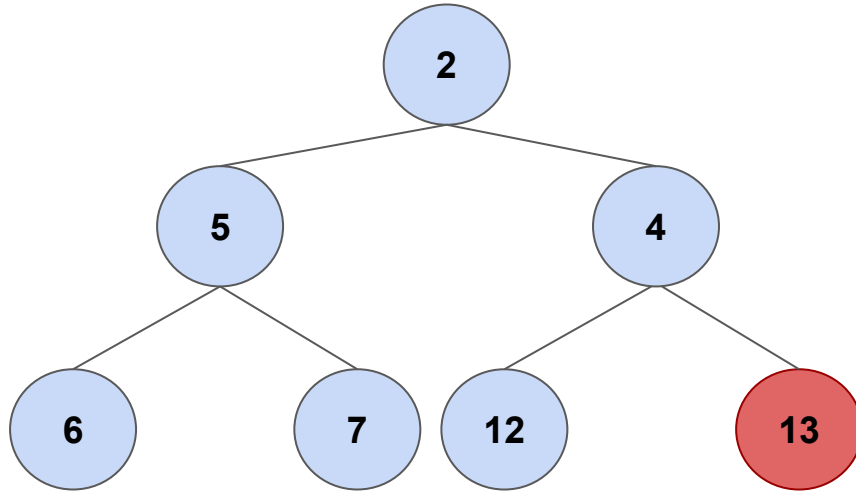
1a



Add to the end



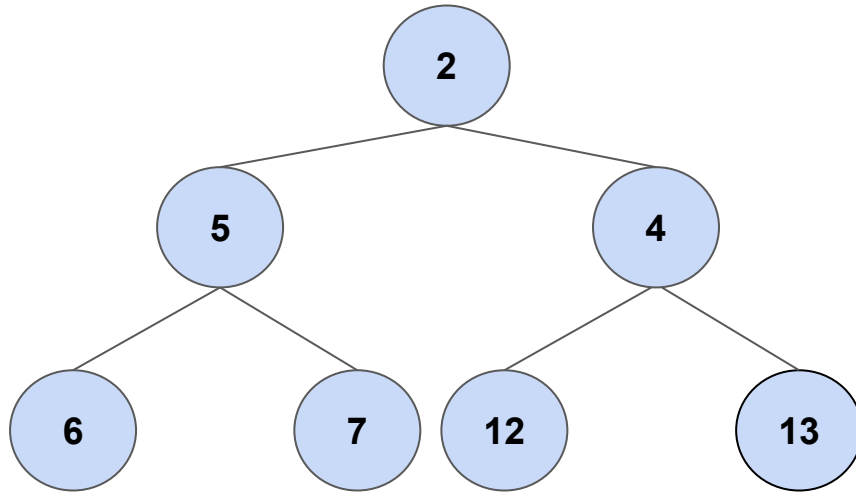
1a



Bubble Up? Nope!



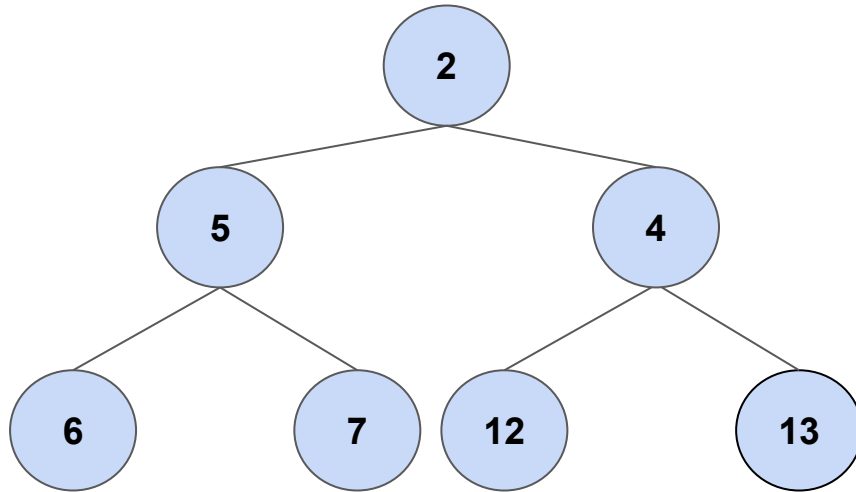
1a



Done!



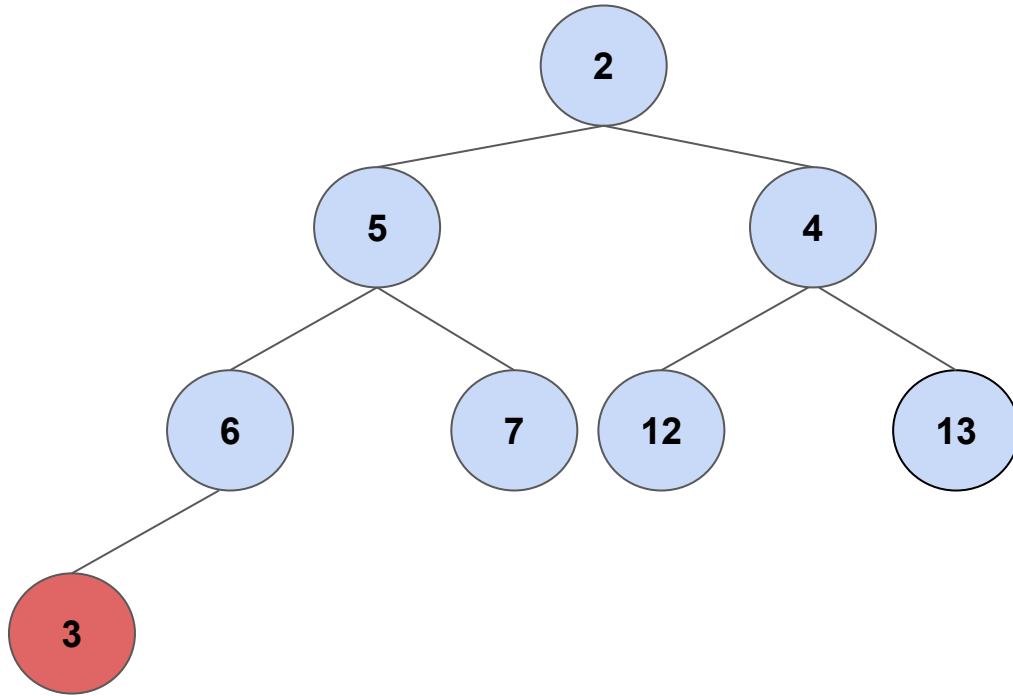
1a



add(3)



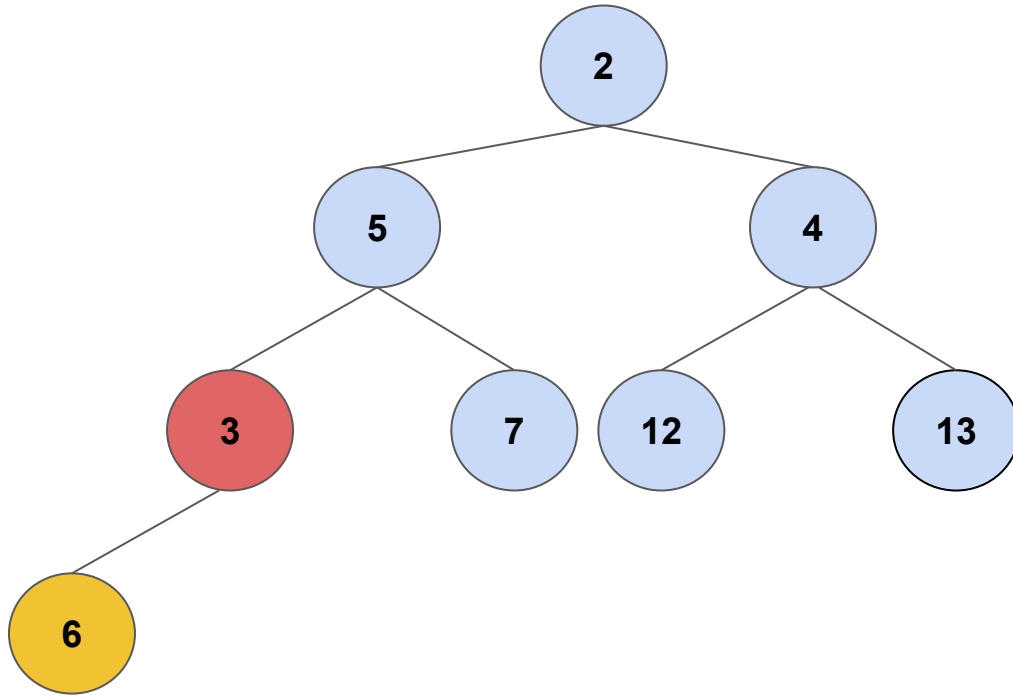
1a



Add to the end



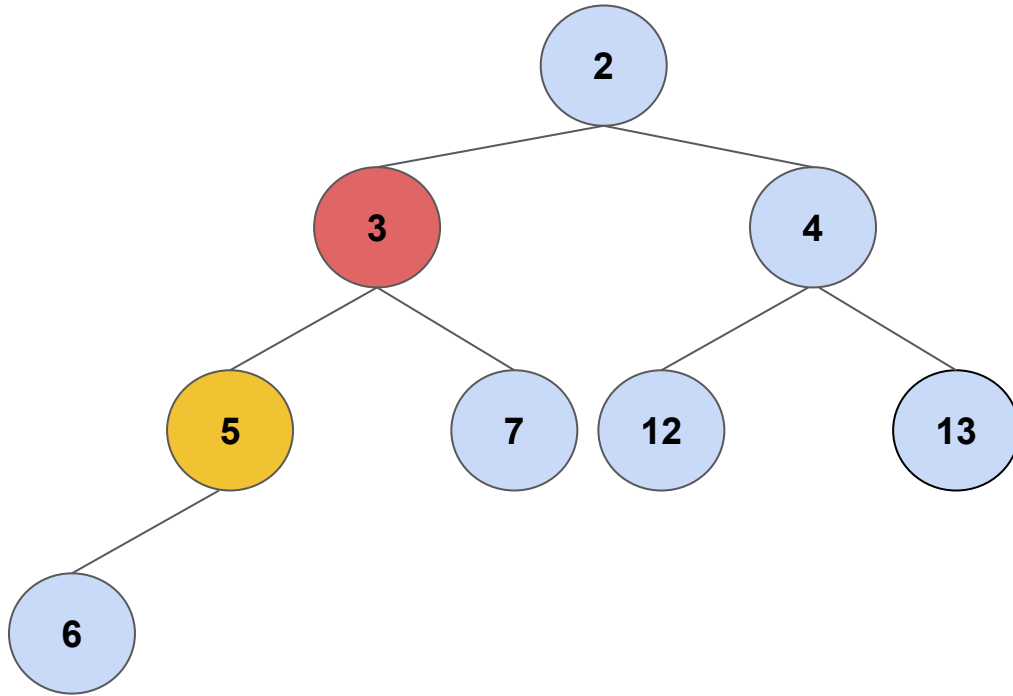
1a



Bubble Up!



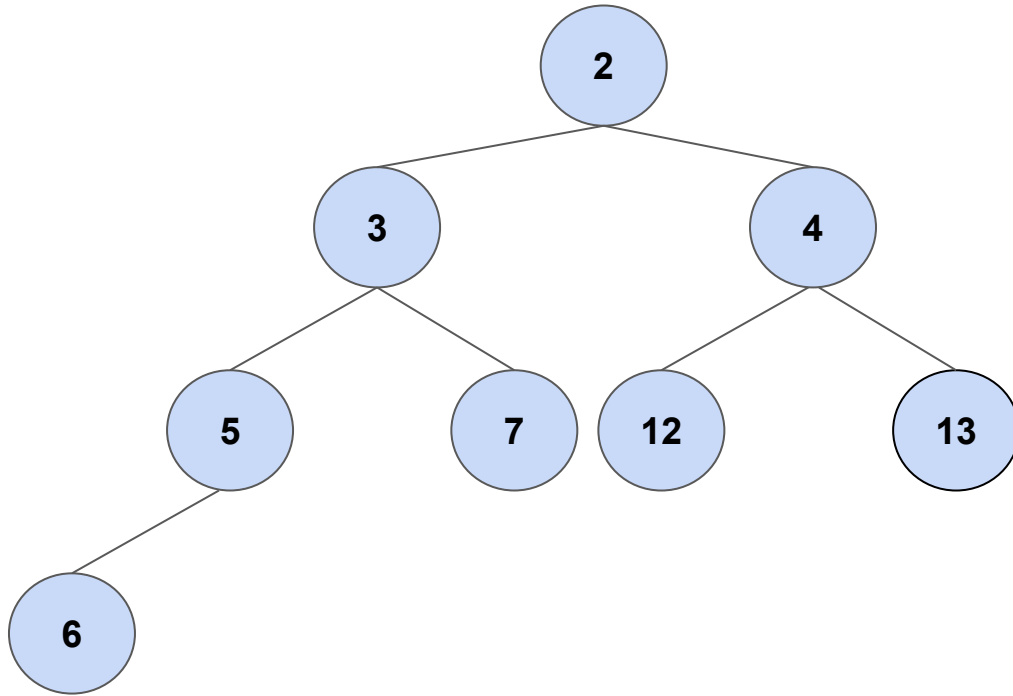
1a



Bubble Up!



1a



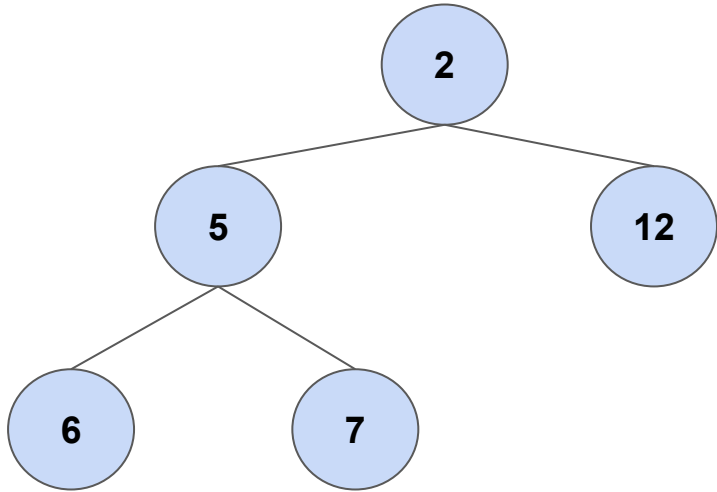
Done!



1b



1b



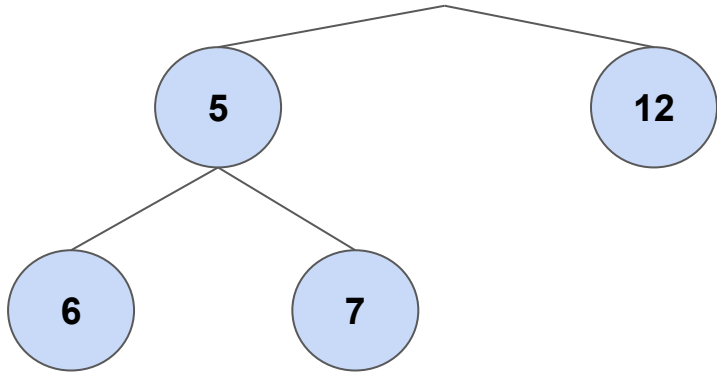
Consider this heap again.

Draw the heap after 2
calls of removeMin.

(after removing smallest
element twice)



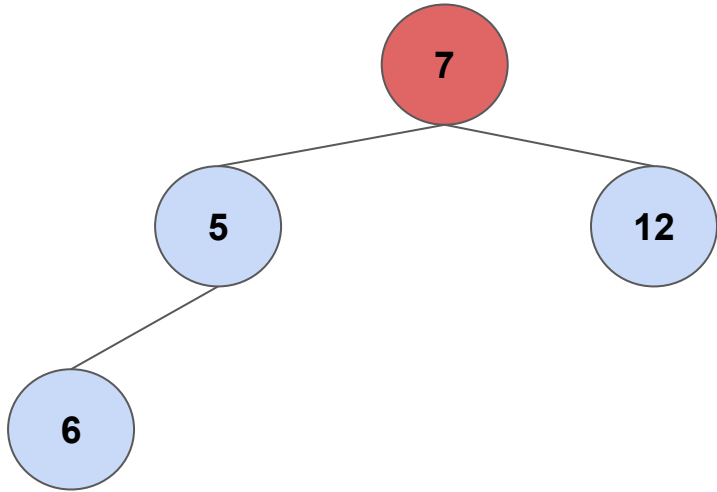
1b



Remove smallest for the
1st time



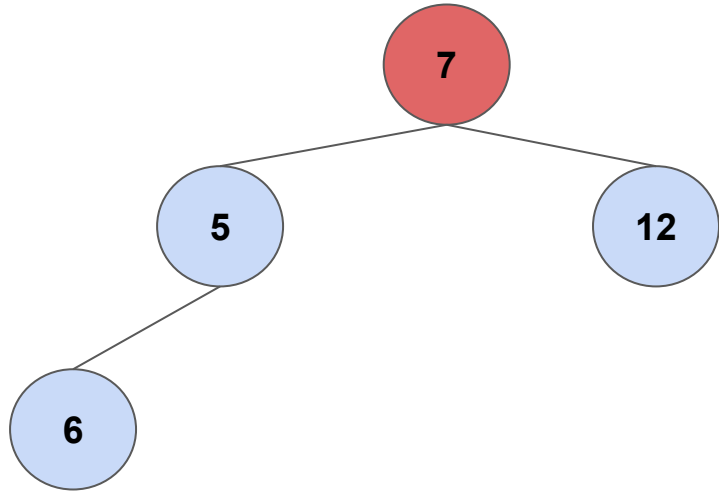
1b



Promote last element!



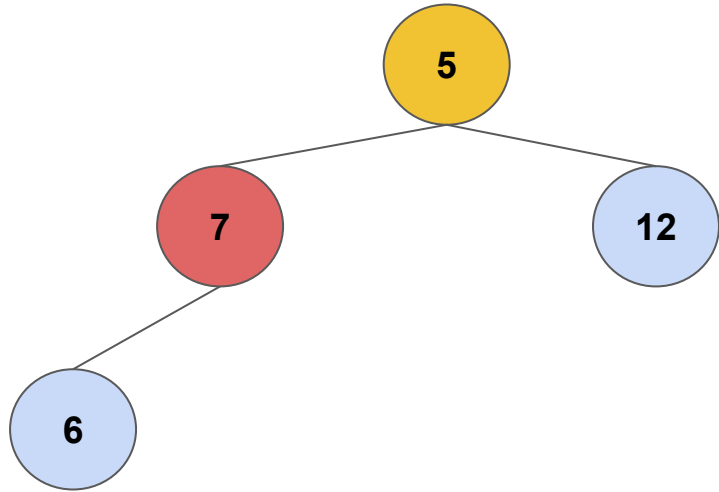
1b



Bubble Down!



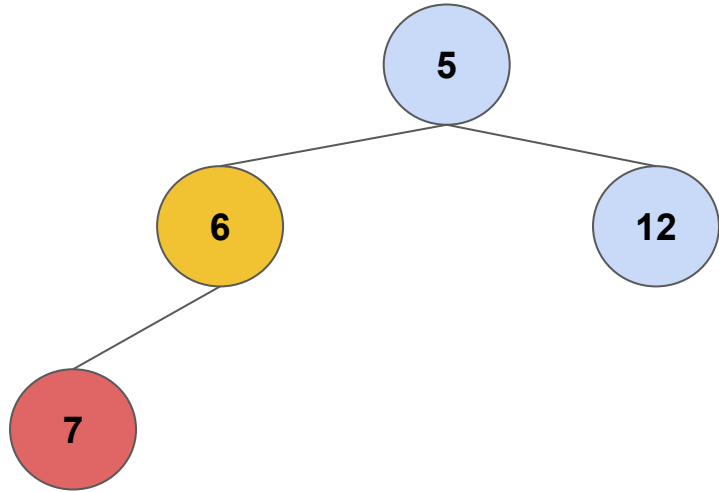
1b



Bubble Down!



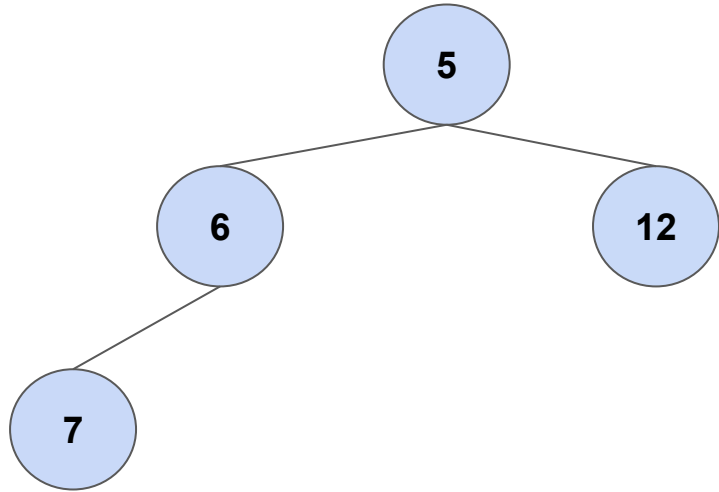
1b



Bubble Down!



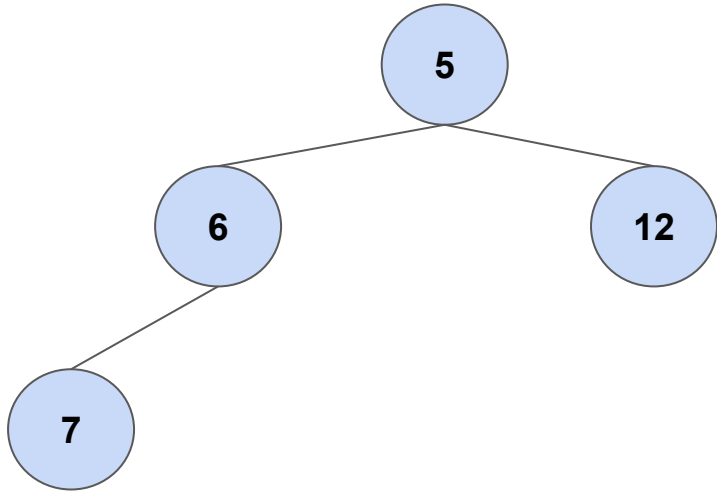
1b



Done!



1b

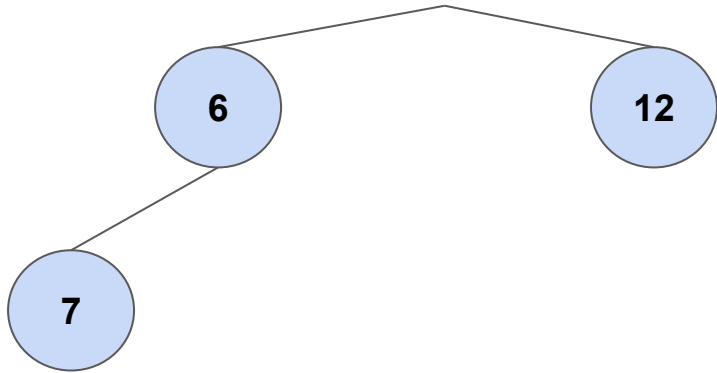


**Remove smallest element
for the 2nd time!**

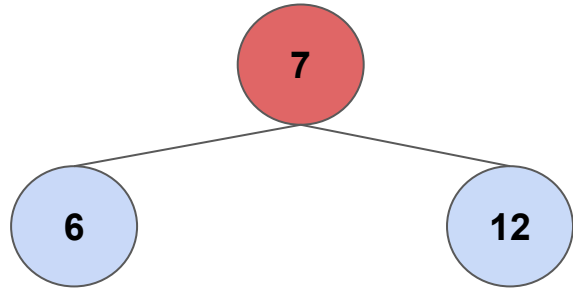


1b

Remove root!



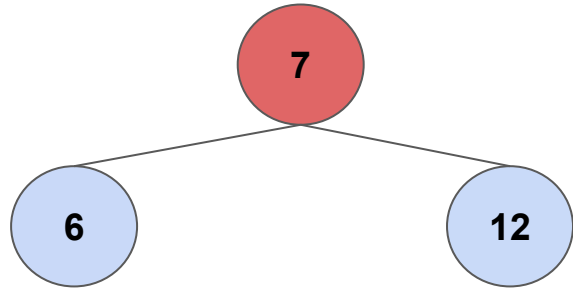
1b



Promote last element!



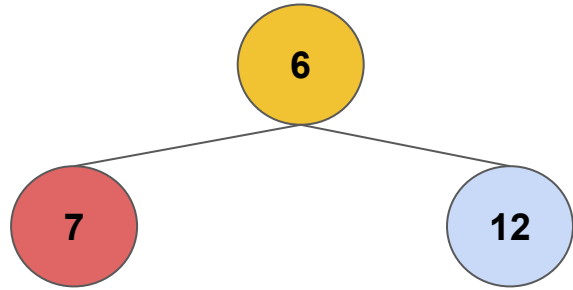
1b



Bubble Down!



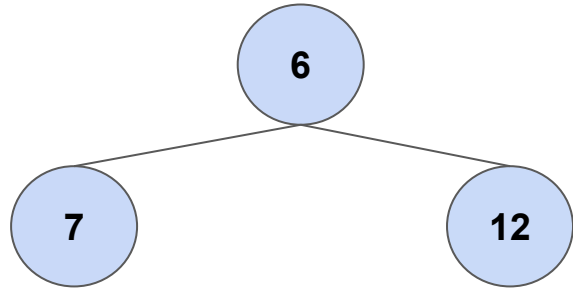
1b



Bubble Down!



1b



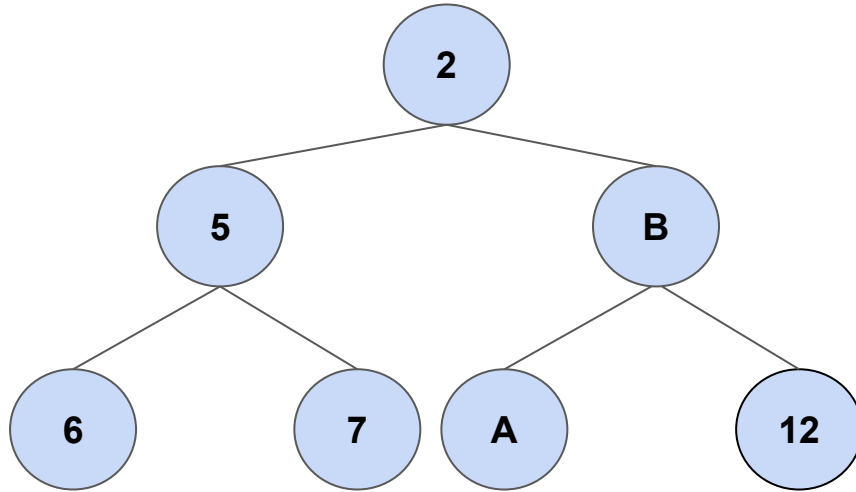
Done!



1c



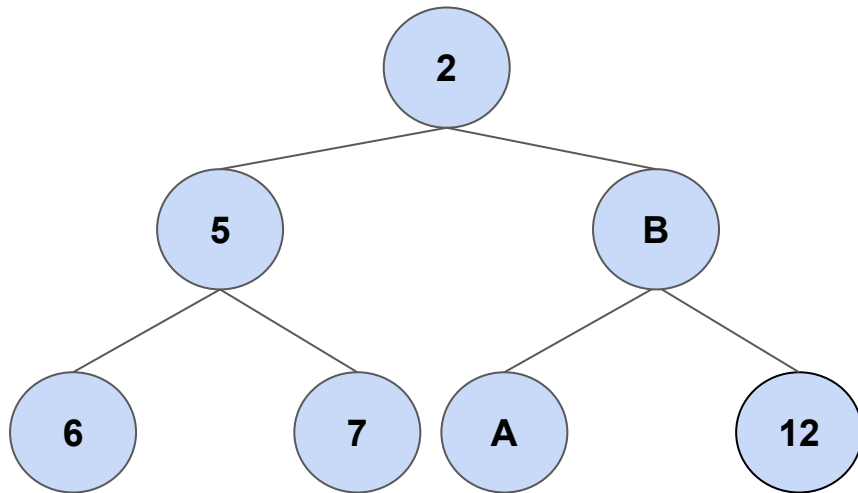
1c



Consider this heap after inserting unknowns A and B.



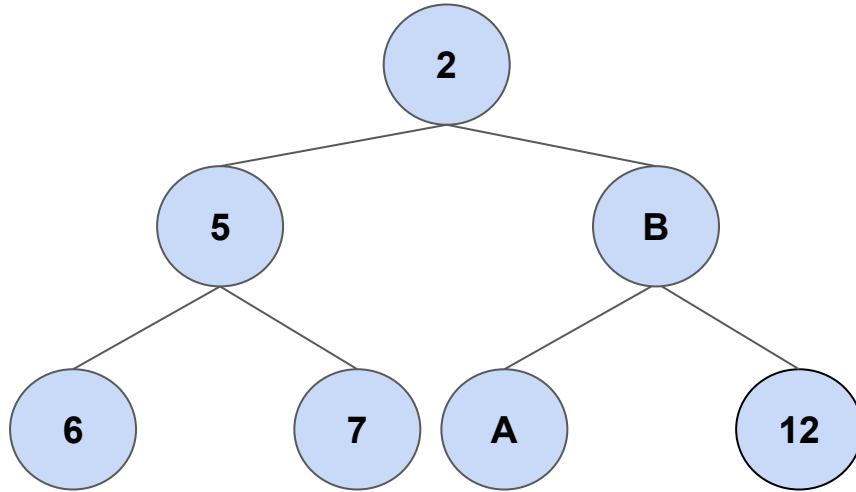
1c



Is $A < B$ true?



1c



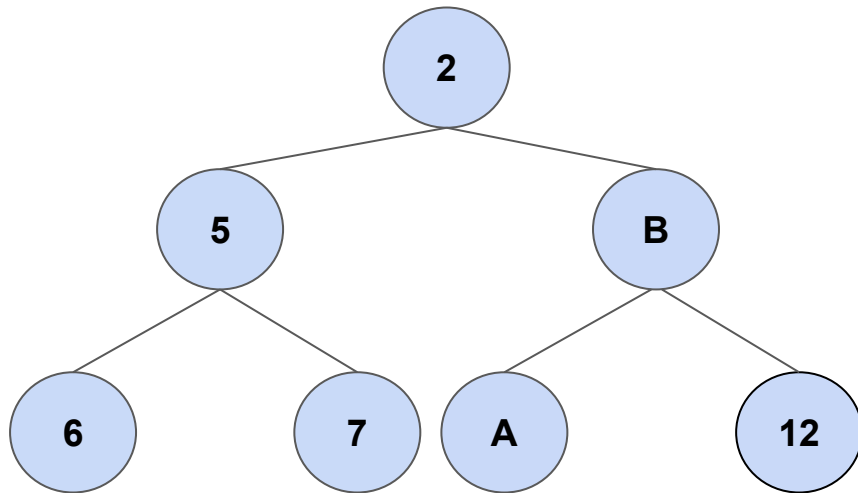
Is $A < B$ true?

Never. This is a min-heap.

Child $\geq$ parent, $A \geq B$



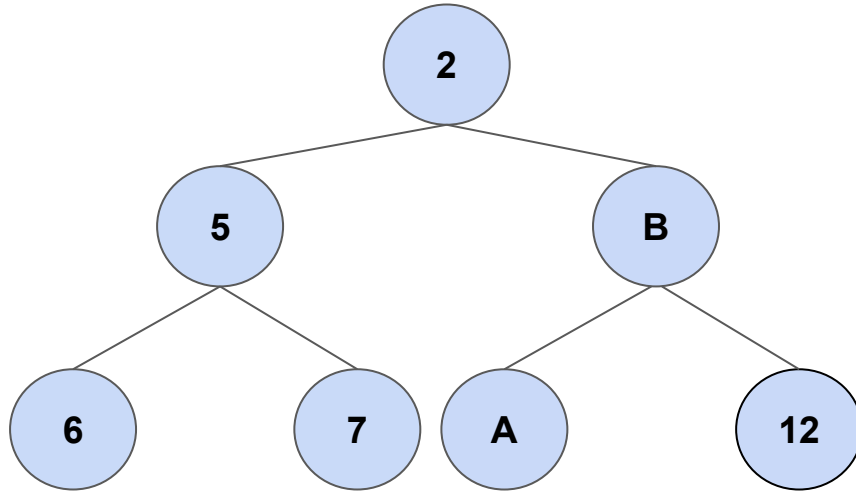
1c



Is $B > 5$ true?



1c



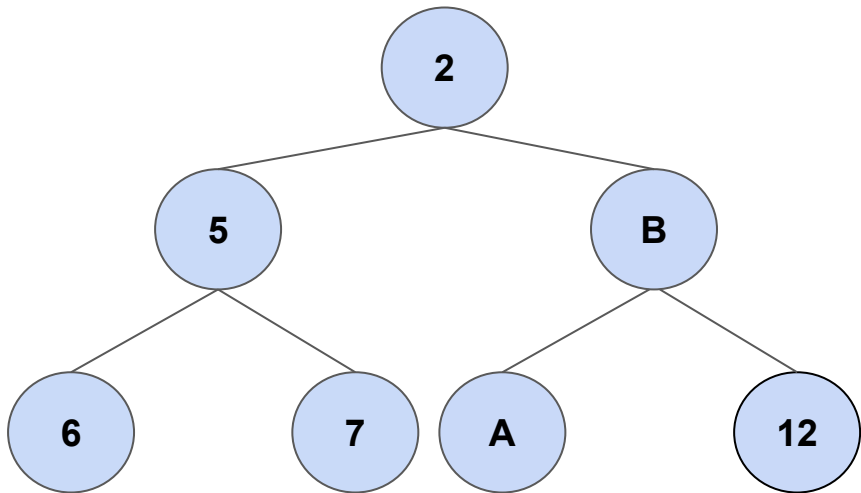
Is $B > 5$ true?

Sometimes. Not enough info comparing 2 nodes at the same depth. Only parent-child node relationships give you information.



1c

Is $A \geq 12$ true?



Consider this heap after inserting unknowns A and B.

$2 \leq B$

$B \leq A$

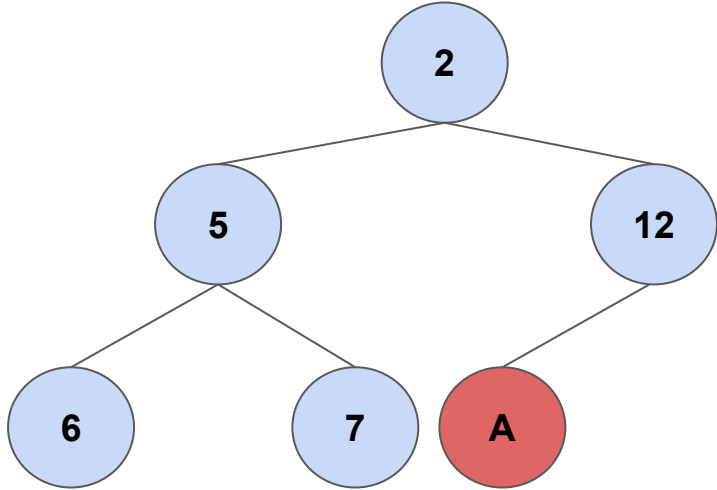
$B \leq 12$

Let's see how we got here.



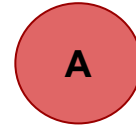
1c

Is $A \geq 12$ true?



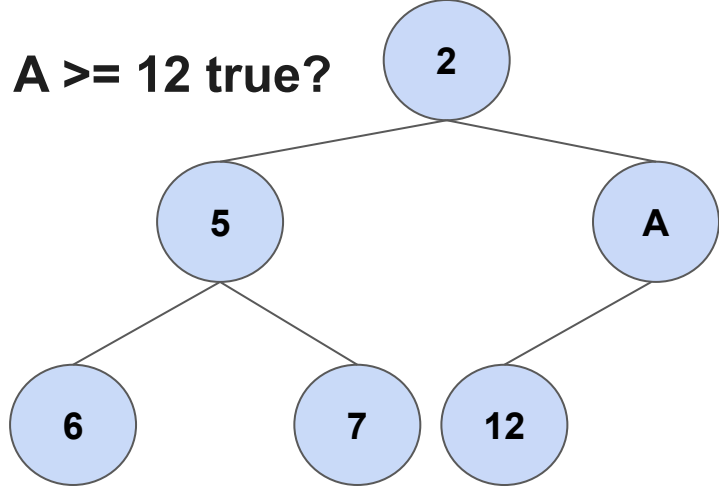
From original heap:

add(A)



1c

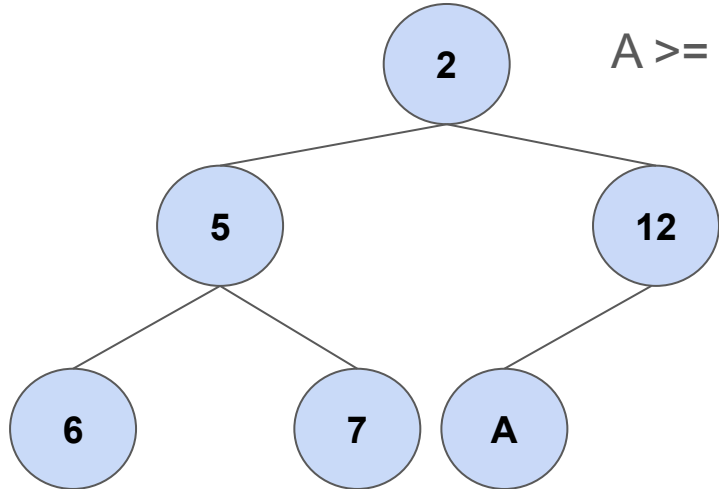
Is $A \geq 12$ true?



$A \leq 12$

$A ? 12$

We end up with 2 possible
heaps.



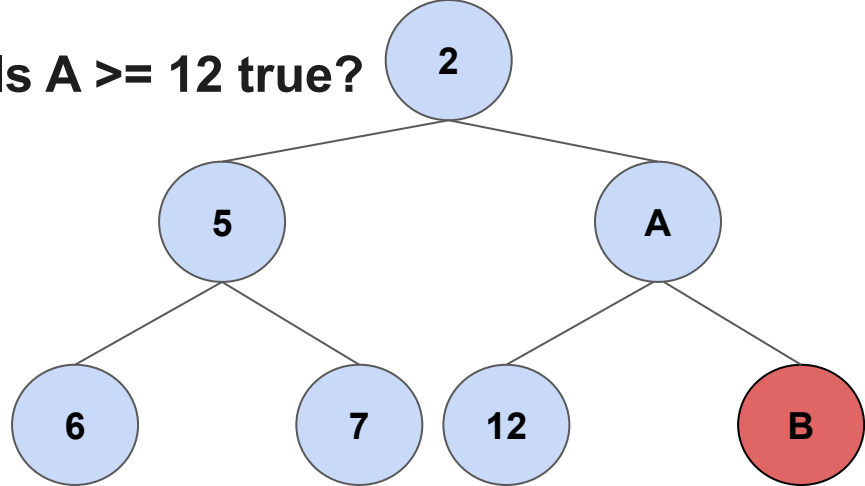
$A \geq 12$

Let's see which one gets
us to our resulting heap.

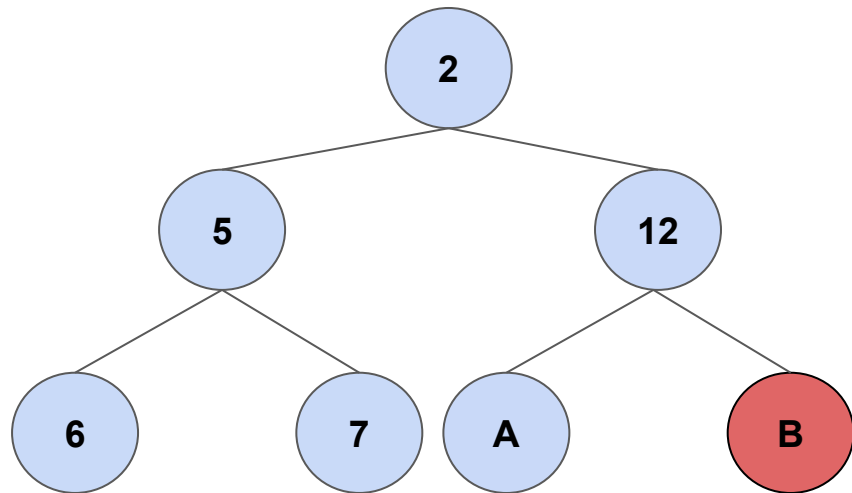
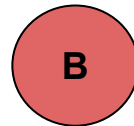


1c

Is $A \geq 12$ true?

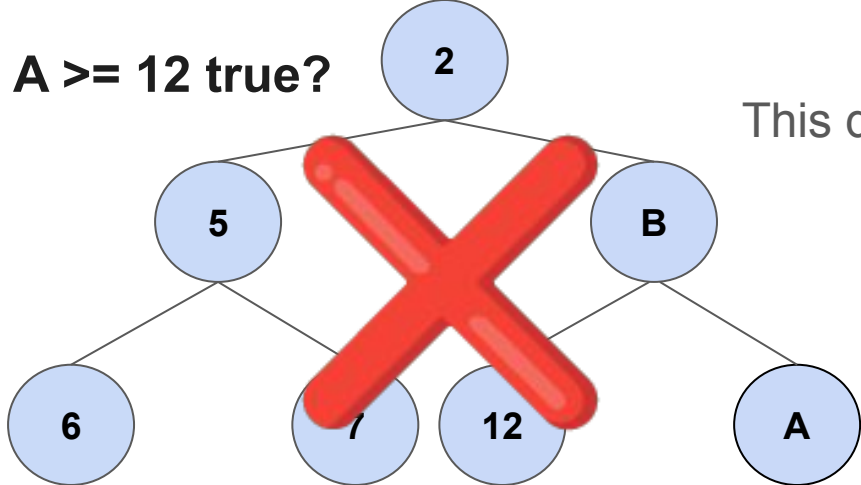


add(B)



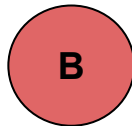
1c

Is $A \geq 12$ true?



This doesn't match our goal

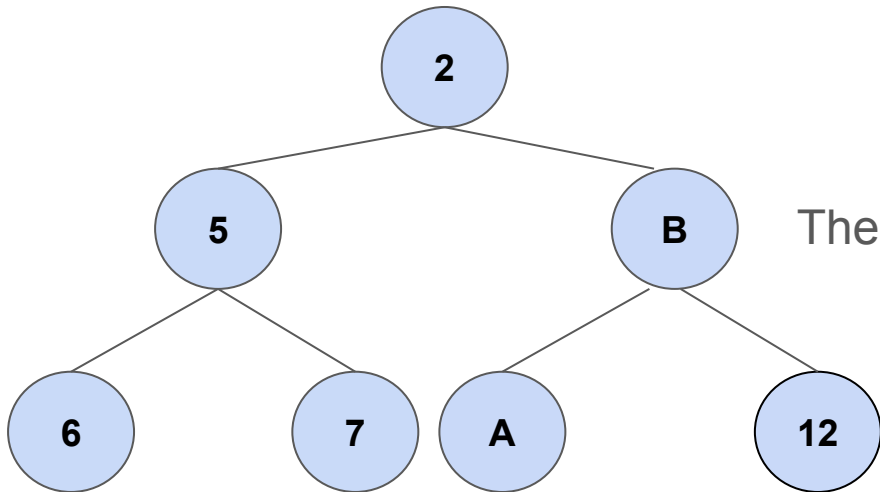
add(B)



$2 \leq B$

$B \leq A$

$B \leq 12$



Therefore, $12 \leq A \rightarrow \text{ALWAYS}$



1d



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

1. `removeMin` has a best case runtime of _____ and a worst case runtime of _____.
2. `insert` has a best case runtime of _____ and a worst case runtime of _____.
3. A _____ or _____ traversal on a min-heap *may* output the elements in sorted order. Assume there are at least 3 elements in the min-heap.
4. The fourth smallest element in a min-heap with 1000 elements can appear in _____ places in the heap.
5. Given a min-heap with $2^N - 1$ distinct elements, for an element
 - to be on the second level it must be less than _____ element(s) and greater than _____ element(s).
 - to be on the bottommost level it must be less than _____ element(s) and greater than _____ element(s).

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

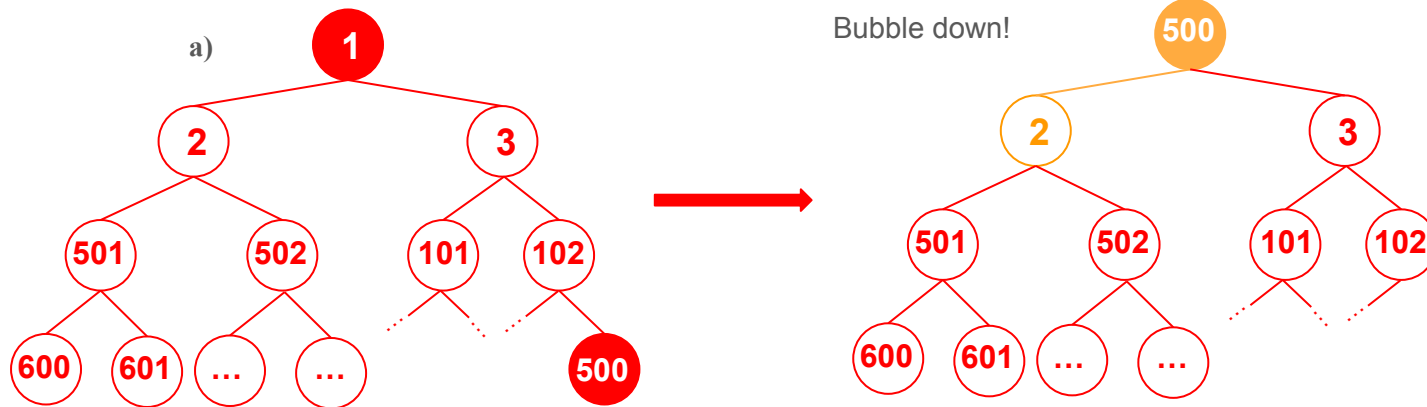
1. `removeMin` has a best case runtime of _____^a and a worst case runtime of _____^b.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

1. `removeMin` has a best case runtime of $\Theta(1)$ and a worst case runtime of _____.

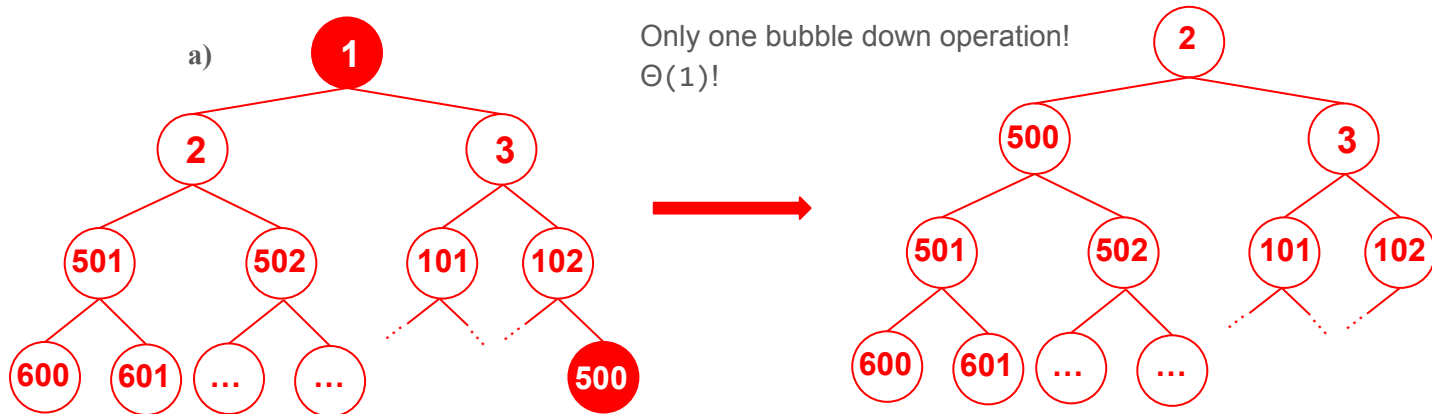


1d

2 Fill in the Blanks

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

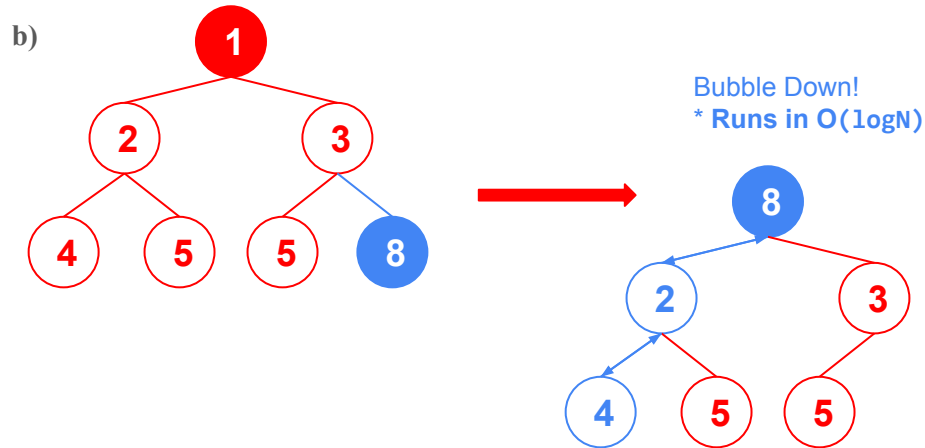
1. `removeMin` has a best case runtime of $\Theta(1)$ and a worst case runtime of _____.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

1. `removeMin` has a best case runtime of $\Theta(1)$ and a worst case runtime of $\Theta(\log N)$.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

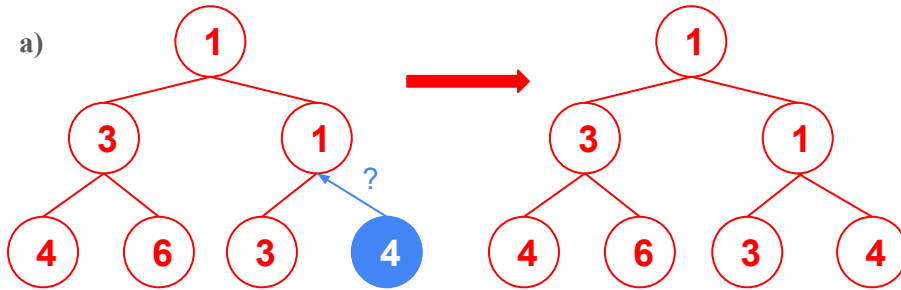
2. `insert` has a best case runtime of _____^a and a worst case runtime of _____^b.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

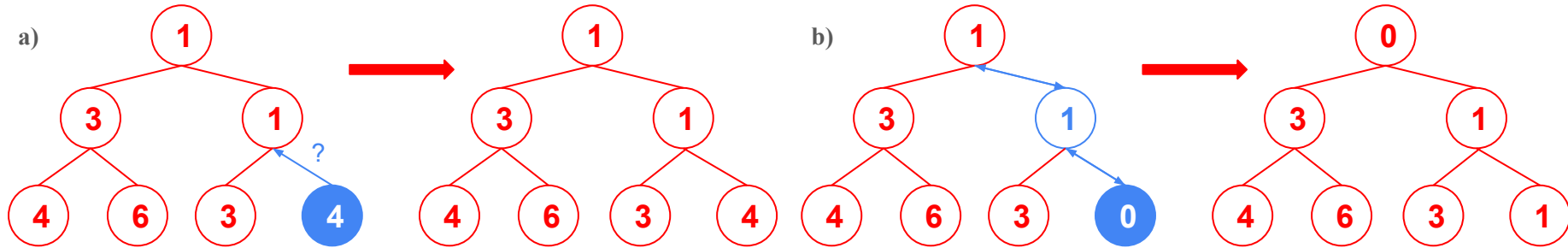
2. `insert` has a best case runtime of $\Theta(1)$ and a worst case runtime of $\Theta(\log N)$.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

2. `insert` has a best case runtime of $\Theta(1)$ and a worst case runtime of $\Theta(\log N)$.

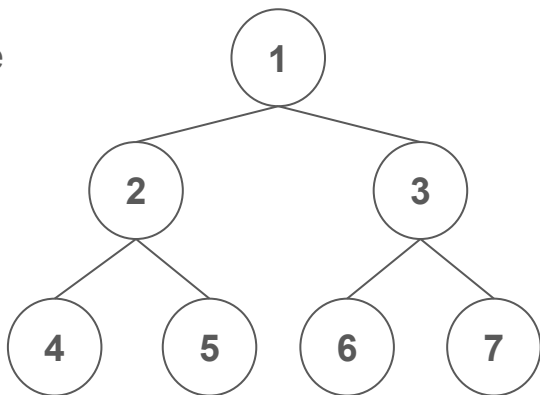


1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

1. `removeMin` has a best case runtime of $\Theta(1)$ and a worst case runtime of $\Theta(\log N)$.
2. `insert` has a best case runtime of $\Theta(1)$ and a worst case runtime of $\Theta(\log N)$.
3. A _____ or _____ traversal on a min-heap *may* output the elements in sorted order. Assume there are at least 3 elements in the min-heap.

Consider the following Min-Heap.



In-Order Traversal

--	--	--	--	--	--	--

Pre-Order Traversal

--	--	--	--	--	--	--

Post-Order Traversal

--	--	--	--	--	--	--

Level-Order Traversal

--	--	--	--	--	--	--

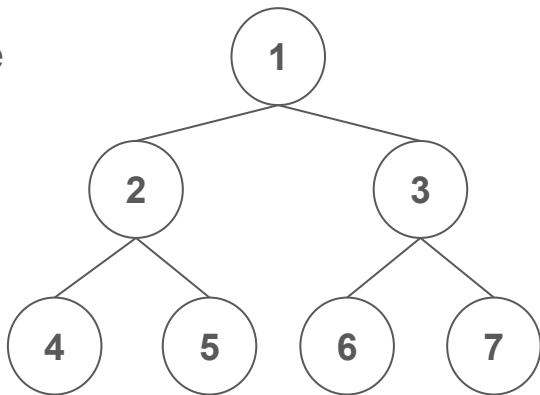


1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

3. A Pre-Order Traversal or Level-Order Traversal traversal on a min-heap *may* output the elements in sorted order. Assume there are at least 3 elements in the min-heap.

Consider the following Min-Heap.



In-Order Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Pre-Order Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Post-Order Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---

Level-Order Traversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---

In this example, while still a valid answer, **pre-order** doesn't necessarily output the elements in order.

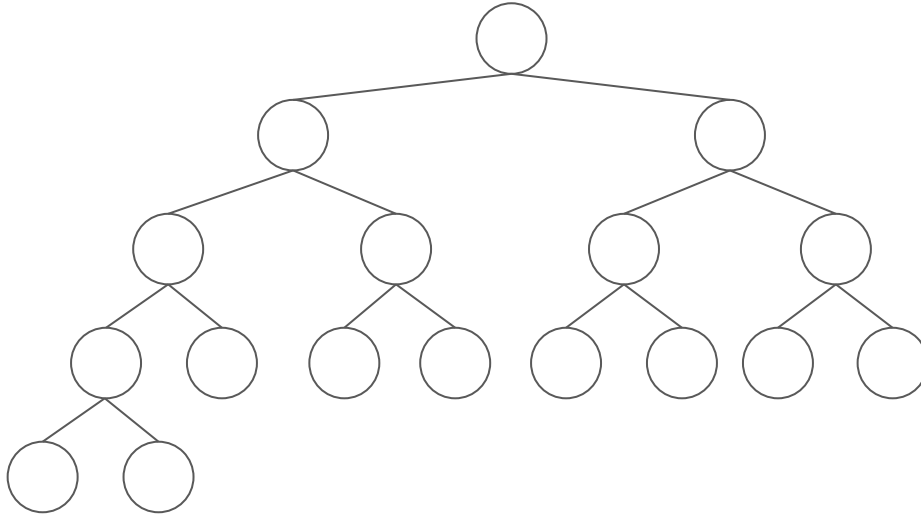
Try drawing an example that **would**, yourself.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

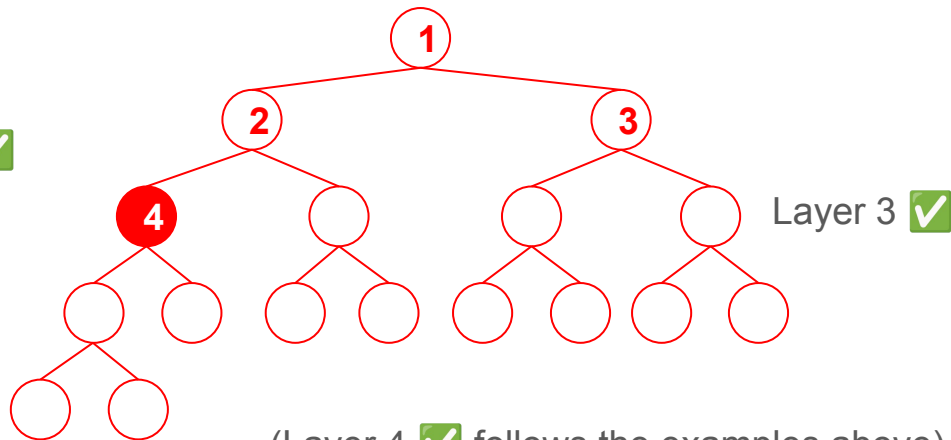
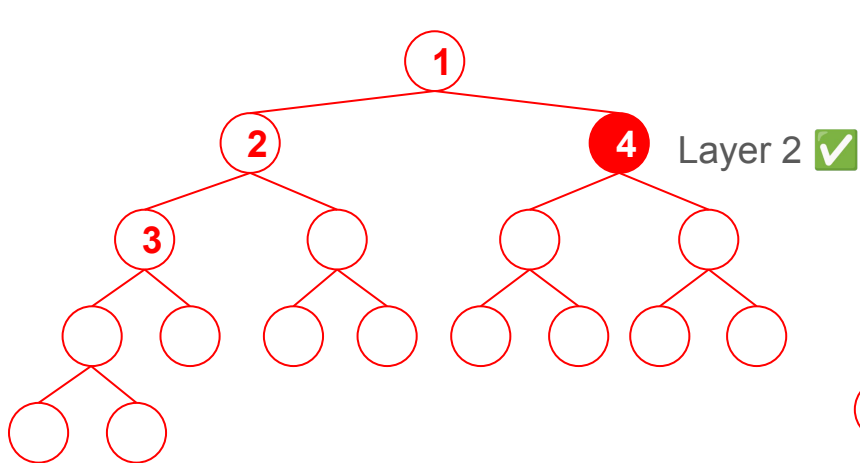
4. The fourth smallest element in a min-heap with 1000 elements can appear in _____ places in the heap.



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

4. The fourth smallest element in a min-heap with 1000 elements can appear in _____ places in the heap.



(Layer 4 ☒ follows the examples above)

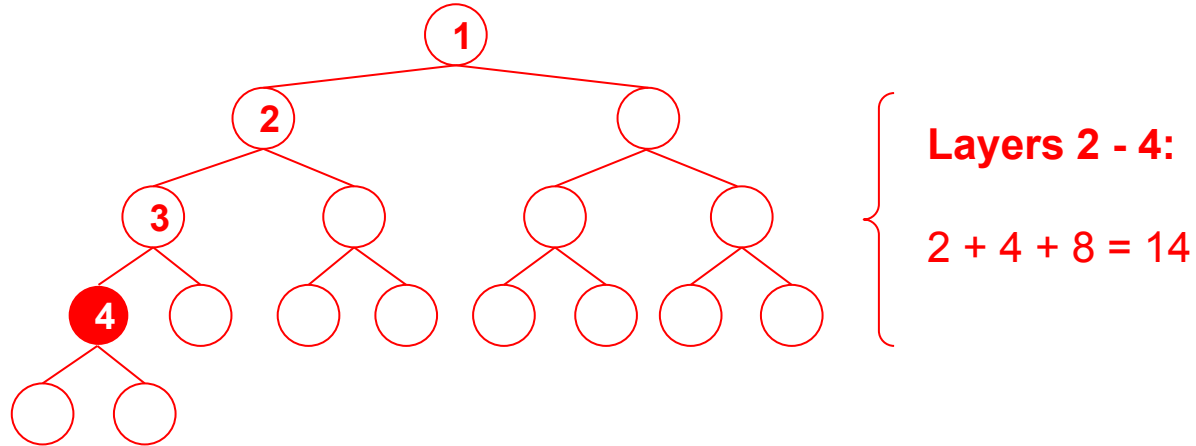
If our 4th smallest value can live in a node in layer x , then it can exist in **any** node across that same layer. Try drawing out more examples yourself!



1d

Fill in the following blanks related to min-heaps. Let N be the number of elements in the min-heap. For the entirety of this question, assume the elements in the min-heap are **distinct**.

4. The fourth smallest element in a min-heap with 1000 elements can appear in _____ **14** _____ places in the heap.



The 4th smallest element can only appear as deep as the 4th layer. To find the number of nodes per level you can do $2^{(\text{layer}-1)}$. For example, root has 2^0 nodes, 2nd layer has 2^1 nodes, and 4th layer has 2^3 nodes.



1d

5. Given a min-heap with $2^N - 1$ distinct elements, for an element

- to be on the second level it must be less than _____ element(s) and greater than _____ element(s).
- to be on the bottommost level it must be less than _____ element(s) and greater than _____ element(s).

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.



1d

5. Given a min-heap with $2^N - 1$ distinct elements, for an element

- to be on the second level it must be less than _____ element(s) and greater than _____ element(s).
- to be on the bottommost level it must be less than _____ element(s) and greater than _____ element(s).

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.

As an example, consider the complete binary tree with $2^3 - 1$ (i.e. $N=3$) elements.

- Containing $2^3 - 1 = 7$ Nodes total, meaning we'll have a complete tree.

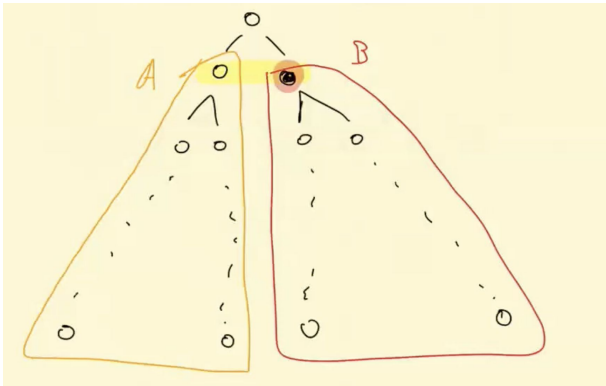


1d

5. Given a min-heap with $2^N - 1$ distinct elements, for an element

- to be on the second level it must be less than $2^{(N-1)} - 2$ element(s) and greater than 1 element(s).

Strategy: partition our Heap into 2 equal sub-Heaps



$2^N - 2$ total elements in A and B

$(2^N - 2) / 2$ elements in just A or B.
Simplifies to $2^{N-1} - 1$

$\Rightarrow$ Larger than $2^{N-1} - 2$ elements

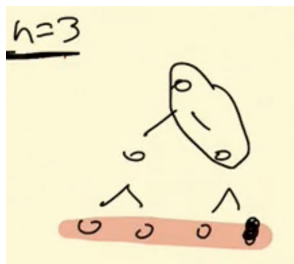


1d

5. Given a min-heap with $2^N - 1$ distinct elements, for an element

- to be on the second level it must be less than $2^{(N-1)} - 2$ element(s) and greater than 1 element(s).
- to be on the bottommost level it must be less than 0 element(s) and greater than $N - 1$ element(s).

Hint: A complete binary tree (with a full last-level) has $2^N - 1$ elements, with N being the number of levels.



Hint says: N = number of levels

Consider our **$N=3$** example.

- A node on the bottom level would be larger than its $N-1$ parents

If you're not convinced, try drawing some examples yourself!



2 Heaps, the Hard Way



2

In lecture, we implemented a heap using a backing array. We could also have done so with a tree representation, given below:

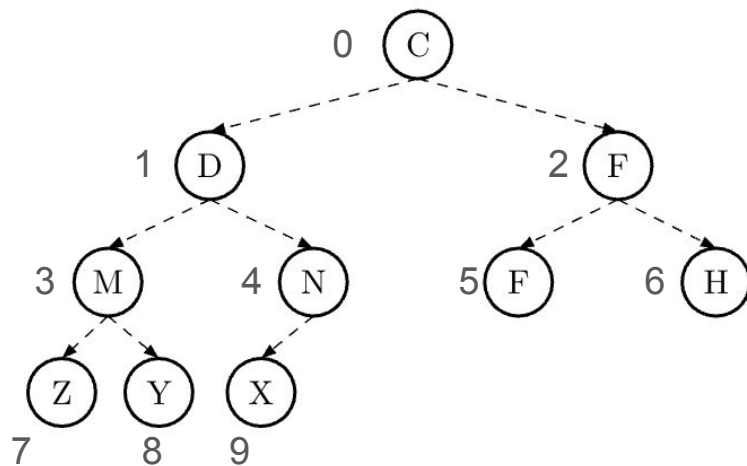
```
private class HeapNode {  
    private T item;  
    private HeapNode left;  
    private HeapNode right;  
  
    HeapNode(T i) {  
        item = i;  
    }  
}
```

One issue with this approach is that it is somewhat tricky to find where to insert new nodes. In this problem, we'll walk through this in some detail.



2a

- (a) Suppose we have a heap that currently looks like the figure below, where the integer beside each node is the “node number”, i.e. position in the heap, of each item.

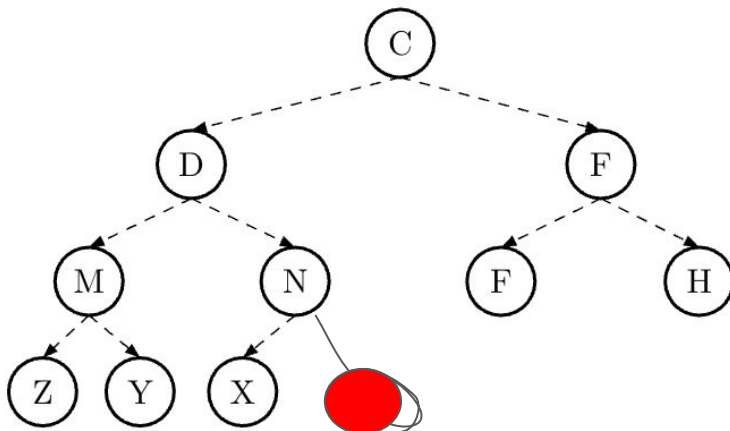


If we add a new item, which node will be the parent of the new node (before doing any **bubbleUp** operations)? Why?



2a

- (a) Suppose we have a heap that currently looks like the figure below, where the integer beside each node is the “node number”, i.e. position in the heap, of each item.



If we add a new item, which node will be the parent of the new node (before doing any **bubbleUp** operations)? Why?

N: completeness property



2b

(b) Fill in the `isRightChild(int i)` and `parent(int i)` methods. Assume for `isRightChild` that `i` is nonnegative and for `parent` that `i` is positive. Notice that the root has node number zero.

- `isRightChild` returns true if the node number `i` corresponds to a right child. E.g. `isRightChild(2)` and `isRightChild(4)` should evaluate to **true**, and `isRightChild(0)` and `isRightChild(1)` should evaluate to **false**
- `parent` returns the node number of the parent of the given node `i`. E.g. `parent(9)` should evaluate to 4

```
private boolean isRightChild(int i) {  
    return
```

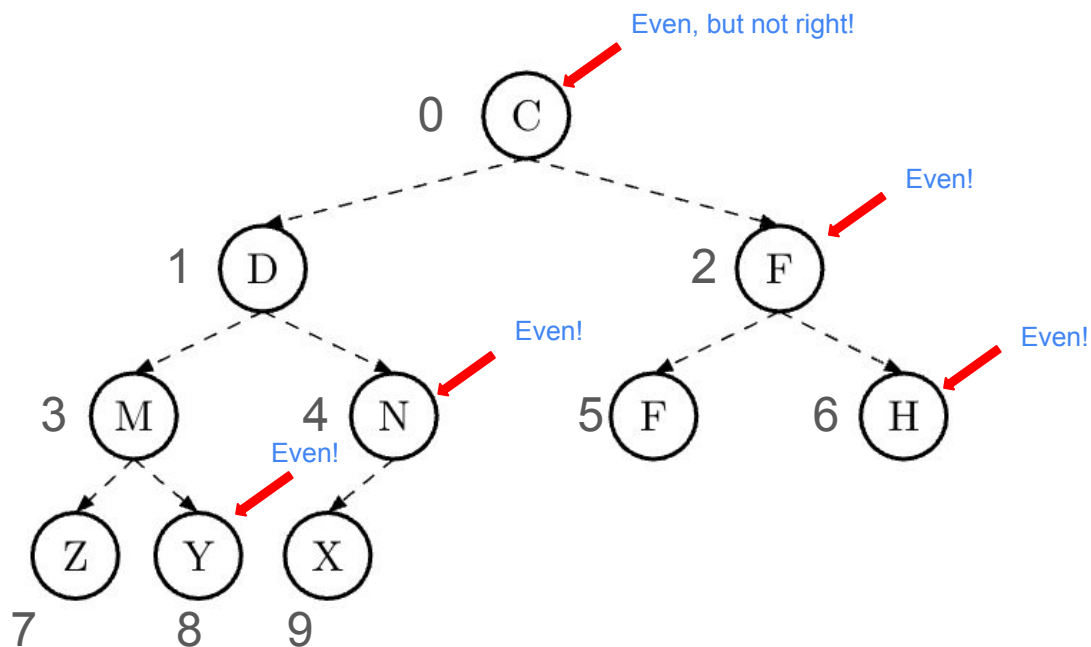
```
_____  
}
```

```
private int parent(int i) {  
    return
```

```
_____  
}
```



2b

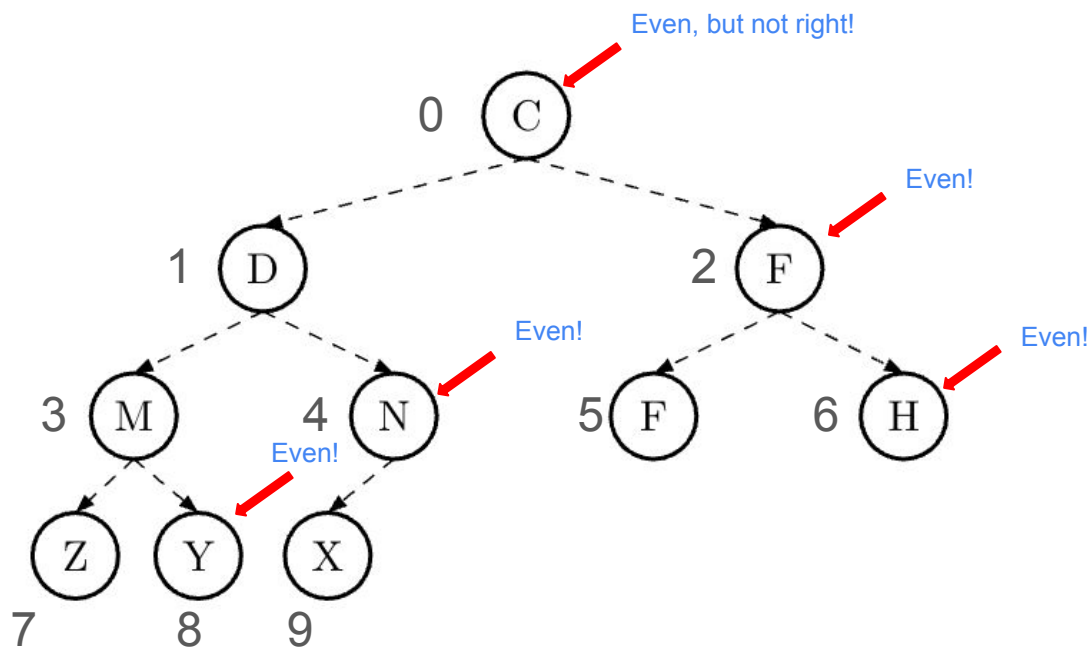


0 1 2 3 4 5 6 7 8 9
[C, D, F, M, N, F, H, Z, Y, X]

1. Notice that all right nodes are even numbers.
2. It must be greater than 0, or else, we include the root!



2b



0 1 2 3 4 5 6 7 8 9
[C, D, F, M, N, F, H, Z, Y, X]

You can also refer to these relations
(0-indexed array representation):

$$\text{.leftChild}(k) = k * 2 + 1$$

$$\text{.rightChild}(k) = (k * 2) + 2$$

$$\text{.parent}(k) = (k-1) // 2$$

Notice that right child must be divisible by 2.



2b

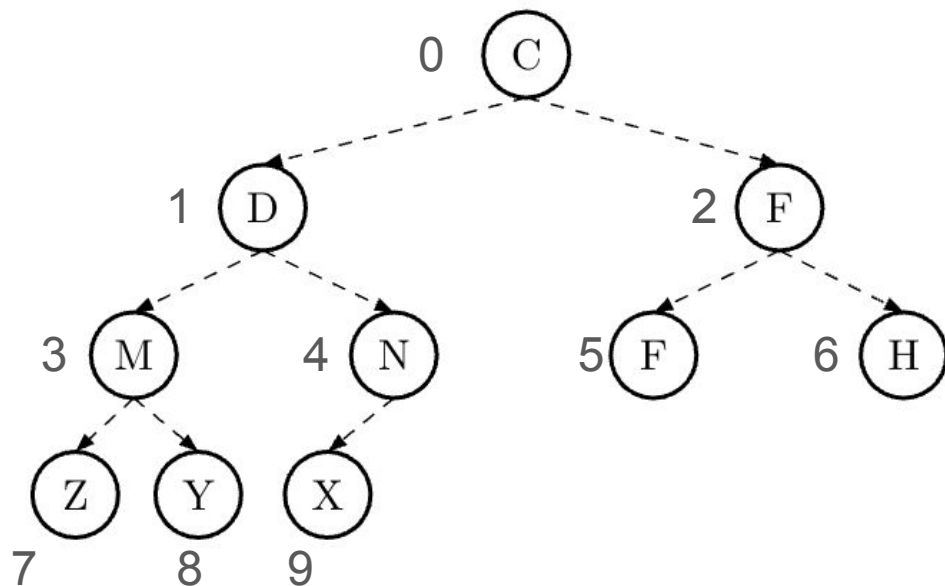
(b) Fill in the `isRightChild(int i)` and `parent(int i)` methods. Assume for `isRightChild` that `i` is nonnegative and for `parent` that `i` is positive. Notice that the root has node number zero.

- `isRightChild` returns true if the node number `i` corresponds to a right child. E.g. `isRightChild(2)` and `isRightChild(4)` should evaluate to `true`, and `isRightChild(0)` and `isRightChild(1)` should evaluate to `false`
- `parent` returns the node number of the parent of the given node `i`. E.g. `parent(9)` should evaluate to 4

```
private boolean isRightChild(int i) {  
    return      i % 2 == 0 && i > 0;  
};  
  
private int parent(int i) {  
    return  
};
```



2b



What is the parent node of 7?

3!

What about for 5?

2!

Hmm.. is there a pattern?

$$(7-1) / 2 = 3$$

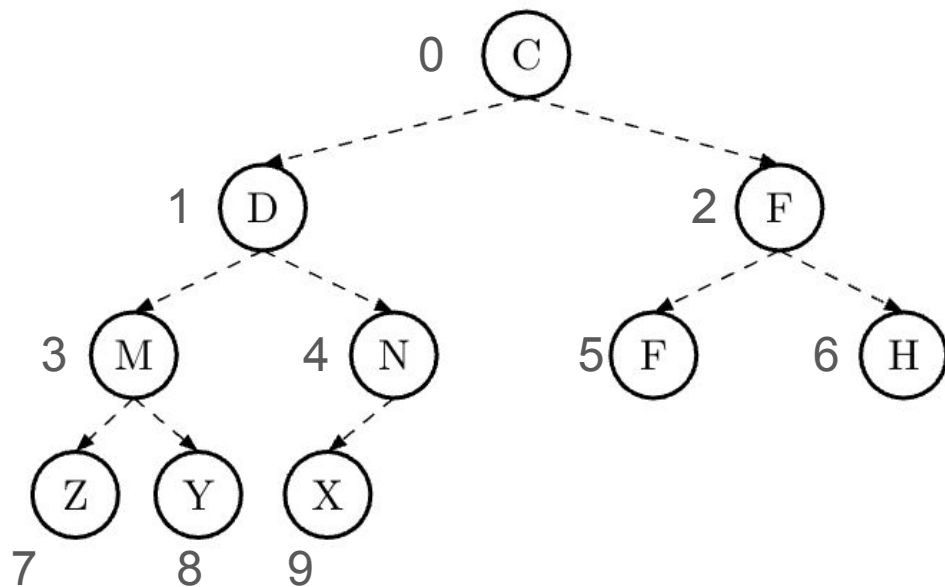
$$(5-1) / 2 = 2$$

What about 8?

$$(8-1) / 2 = 3.5 \Rightarrow \text{floor}(3.5) = 3$$

In Java, integer casting is done by removing parts after decimal point!

2b



You can also refer to these relations
(0-indexed array representation):

$$\text{.leftChild}(k) = k * 2 + 1$$

$$\text{.rightChild}(k) = (k * 2) + 2$$

$$\text{.parent}(k) = (k-1) // 2$$

We can see that $\text{parent}(k)$ is given by $(k - 1) / 2$

2b

(b) Fill in the `isRightChild(int i)` and `parent(int i)` methods. Assume for `isRightChild` that `i` is nonnegative and for `parent` that `i` is positive. Notice that the root has node number zero.

- `isRightChild` returns true if the node number `i` corresponds to a right child. E.g. `isRightChild(2)` and `isRightChild(4)` should evaluate to **true**, and `isRightChild(0)` and `isRightChild(1)` should evaluate to **false**
- `parent` returns the node number of the parent of the given node `i`. E.g. `parent(9)` should evaluate to 4

```
private boolean isRightChild(int i) {  
    return      i % 2 == 0 && i > 0;;  
}  
  
private int parent(int i) {  
    return      (i - 1) / 2;;  
}
```



2c

- (c) Fill in the followPath(List path) method so that it matches the given javadoc comment. Assume the given path is valid.

```
private HeapNode followPath(List<Boolean> path) {  
    HeapNode p = root;  
  
    for (_____ : _____){  
  
        if (_____) {  
  
            _____;  
  
        } else {  
  
            _____;  
  
        }  
    }  
    return p;  
}
```



2c

True: right, false: left for entire path

- (c) Fill in the followPath(List path) method so that it matches the given javadoc comment. Assume the given path is valid.

```
private HeapNode followPath(List<Boolean> path) {  
    HeapNode p = root;  
    for ( boolean b : path ){  
        if ( b ) {  
            p = p.right;  
        } else {  
            p = p.left;  
        }  
    }  
    return p;  
}
```



2d

- (d) The method below will return the i^{th} node. For example, if we call `getNode(9)` on the example heap, we will get the HeapNode containing X.

```
private HeapNode getNodeI(int i) {  
    List<Boolean> pathToI = constructPath(i);  
    HeapNode nodeNumberI = followPath(pathToI);  
    return nodeNumberI;  
}
```

Fill in the `constructPath` method so that the `getNodeI` method above works correctly. You may use any method from the previous parts of this problem. *Hint: Recall that `LinkedList` has methods `addFirst`, `addLast`, `removeFirst`, and `removeLast`.*

```
private List<Boolean> constructPath(int i) {  
    LinkedList<Boolean> list = new LinkedList<>();  
  
    while (_____) {  
  
        _____;  
  
        _____;  
  
    }  
}  
return list;  
}
```

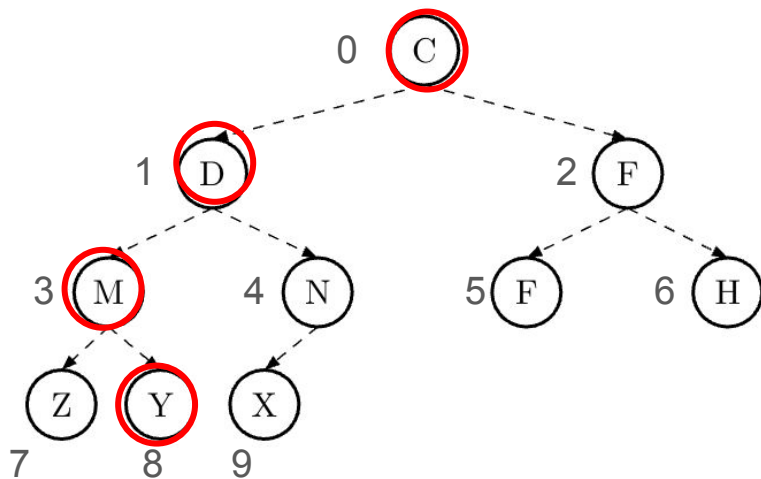


2d

- (d) The method below will return the i^{th} node. For example, if we call `getNode(9)` on the example heap, we will get the HeapNode containing X.

```
private HeapNode getNodeI(int i) {
    List<Boolean> pathToI = constructPath(i);
    HeapNode nodeNumberI = followPath(pathToI);
    return nodeNumberI;
}
```

Fill in the `constructPath` method so that the `getNodeI` method above works correctly. You may use any method from the previous parts of this problem. *Hint: Recall that `LinkedList` has methods `addFirst`, `addLast`, `removeFirst`, and `removeLast`.*



- Construct path from i to root
- Record if step taken was left or right
- Add elems from right to left (using `addFirst`)

`constructPath(8) = [false, false, true]`



2d

- (d) The method below will return the i^{th} node. For example, if we call `getNode(9)` on the example heap, we will get the HeapNode containing X.

```
private HeapNode getNodeI(int i) {  
    List<Boolean> pathToI = constructPath(i);  
    HeapNode nodeNumberI = followPath(pathToI);  
    return nodeNumberI;  
}
```

Fill in the `constructPath` method so that the `getNodeI` method above works correctly. You may use any method from the previous parts of this problem. *Hint: Recall that `LinkedList` has methods `addFirst`, `addLast`, `removeFirst`, and `removeLast`.*

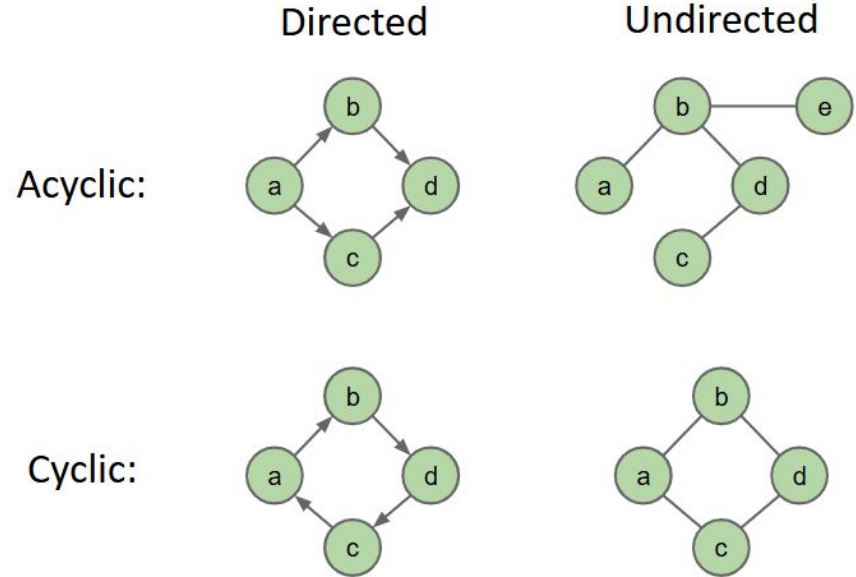
```
private List<Boolean> constructPath(int i) {  
    LinkedList<Boolean> list = new LinkedList<>();  
    while (_____) {  
        _____;  
        _____;  
    }  
    return list;  
}
```



2. Graph Algorithms

What is a graph?

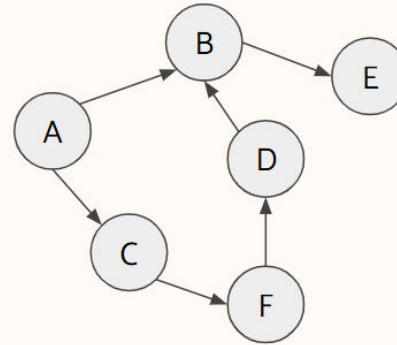
- A collection of “vertices” (nodes) that are connected by “edges” (lines)
- Tree = type of graph
 - Acyclic, N vertices, $N - 1$ edges
- Why graphs?
 - They can represent complex, non-linear information, like your social media network, or a Google Maps request!



How do you represent a graph?

- Adjacency List = maps each vertex to a LIST of neighboring vertices
- Space: $O(V + E)$

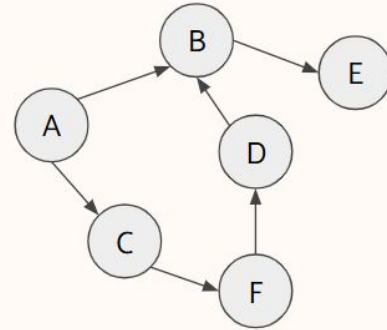
A	B, C
B	E
C	F
D	B
E	
F	D



How do you represent a graph?

- Adjacency Matrix = 2-D Array where edges between vertices are represented as "1"
- Space: It's a 2-D array of fixed length $\rightarrow O(V^2)$

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	0	0	0	0	1	0
C	0	0	0	0	0	1
D	0	1	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	1	0	0



What can you do with a graph?

- By representing real-world phenomena as graphs, we can answer all kinds of problems with graph algorithms
 - Ex: Google Maps might use Dijkstra's
- In this class, we've learned multiple important graph algorithms. They each solve a distinct problem.

GRAPH ALGORITHM CONCEPT MAP:

Start: what Question is being asked?

find a path from A to all other vertices?

DFS

BFS

↳ finds path with minimum # of edges

equivalent if all edges are equally weighted

find shortest path from A to all other vertices? (e.g. a Shortest paths tree)

Dijkstra's

↳ finds path with minimum weight

find a tree that touches all vertices with minimum total weight? (e.g. a minimum spanning tree)

Prim's (bushy) approach

Kruskal's (spindly) approach

How do you find a path from vertex A to B?

- DFS = Depth First Search Algorithm
 - Traverses a graph in a spindly way because it goes as deep as it can
 - Intuition: travel branch by branch
- BFS = Breadth First Search Algorithm
 - Traverses a graph in a bushy way because it radiates outwards from the starting node
 - Intuition: travel layer by layer

DFS Orderings

Multiple ways to implement DFS:

1. **Preorder:** Visit self before children
 - Add a given node to the DFS Traversal Ordering as soon as we see that node
2. **Postorder:** Visit children before self
 - Only add a given node to the DFS Traversal Ordering after backtracking from that node

Runtime: $O(V + E)$

- V = # vertices, E = # edges

How do you find the shortest path from vertex A to B?

- Run Dijkstra's!
 - It's like BFS but works with edge weights
 - Takes in a source node
 - Generates a "shortest paths tree"
 - This tree contains a path from the source node to every other vertex in the graph
- Runtime: $O((V + E) \log E)$

Dijkstra's Breakdown

Dijkstra's Pseudocode

```
1 PQ = new PriorityQueue()
2 PQ.add(A, 0) ← A is the "source"
3 PQ.add(v, infinity) # (all nodes except A).
4
5 distTo = {} # map
6 edgeTo = {} # map
7 distTo[A] = 0
8 distTo[v] = infinity # (all nodes except A).
9
10 while (not PQ.isEmpty()):
11     poppedNode, poppedPriority = PQ.pop()
12
13     for child in poppedNode.children:
14         potentialDist = distTo[poppedNode] +
15             edgeWeight(poppedNode, child)
16
17         if potentialDist < distTo[child]:
18             distTo.put(child, potentialDist)
19             PQ.changePriority(child, potentialDist)
20             edgeTo[child] = poppedNode
```

"fringe"
tracks distance
from A to
every other node
↳ update every time
we find a shorter path

① go through
each
vertex in
fringe

② go through
each neighbor
of the vertex

③ if the
potential distance

to this neighbor is shorter than what's currently mapped in distTo

- update distTo
- draw an edge
between the current
vertex and the

Shortest paths tree
↳ it's an adjacency
map
↳ the output of Dijkstra's

Minimum Spanning Trees

- Has $V - 1$ edges, where V represents the total number of vertices

MST	v.s.	Shortest paths tree
↳ no source ↳ minimize the TOTAL weight of a tree that hits all the vertices		↳ comes from a source (A) ↳ minimize path from Source to each individual vertex in the graph

Why MSTs? Motivating Example

- I'm a road builder who has to connect 10 different buildings on campus. Concrete is expensive, so I want to minimize the amount of concrete I use up on roads.
- If I find the MST between all 10 spots, this tree minimizes the TOTAL amount of concrete I use up building roads!

Ok, so how do I find an MST?

find a tree that
touches all vertices
with minimum total
weight? (e.g. a
minimum spanning
tree)

Prim's
(bushy
approach)

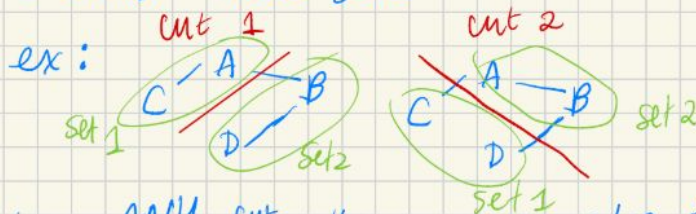
Kruskal's
(spindly
approach)

First, we need to understand...

MST Graph Theory

given a graph, you can split its vertices in 2 sets

↳ cut = the line that splits these sets



- **Cut Property** = given ANY cut, the minimum edge that crosses the cut **BELONGS** in the MST

How does the Cut Property help us find an MST?

Build the MST as you go about this loop:

Loop:

① find a new cut

② add its minimum crossing edge to the MST

Stop if there are $V-1$ edges in the MST

what are Kruskal's & Prim's? Both algorithms accomplish the above in different ways.

Prim's Algorithm

Prim's Breakdown

Prim's Pseudocode

- 1 Start with any node.
- 2 Add that node to the set of nodes in the MST.
- 3 While there are still nodes not in the MST:
- 4 Add the lightest edge from a node in the MST
- 5 that leads to a **new** node that is unvisited
- 6 Add the **new** node to the set of
- 7 nodes in the MST.

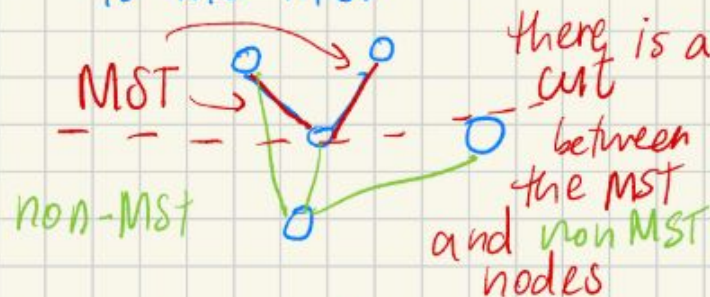
pick the
smallest
edge that
crosses the
MST nodes and

non-MST nodes

NOTICE: in Prim's, we start from one node and radiate outwards, one edge at a time
↳ results in **greedy** MST

Intuition

You're creating a new cut every time you add a new node to the MST



Kruskal's Algorithm

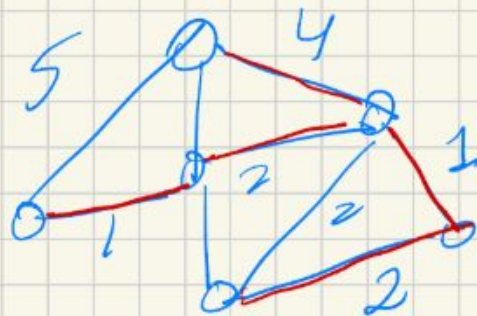
Kruskal's Breakdown

while there
are less
than $V-1$ edges
in the MST

this
is kind
of brute
force

Kruskal's Pseudocode

```
1 while there are still nodes not in the MST:  
2     Add the lightest edge  
3     that does not create a cycle.  
4     Add the new node to the  
5     set of nodes in the MST.
```



Intuition

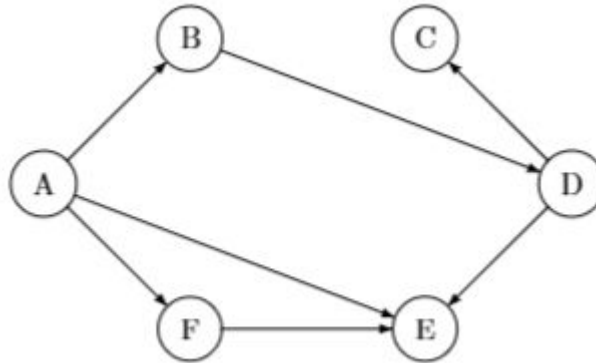
adds the lightest edge that doesn't create a cycle

NOTICE:

Kruskal's results in a spindly MST.

Graph Practice!

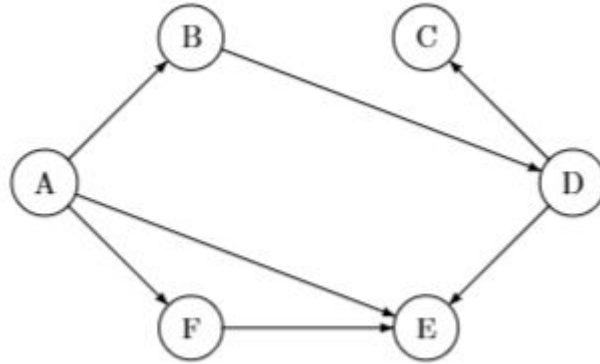
3 Graphs



- (a) Represent the above graph as an adjacency list and adjacency matrix
- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

Solutions:

3 Graphs



(a) Represent the above graph as an adjacency list and adjacency matrix

A → [B, E, F]

B → [D]

C → []

D → [C, E]

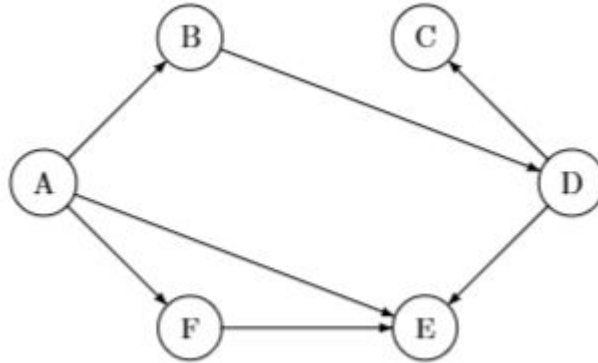
E → []

F → [E]

Note: We follow the convention that nodes do not have an edge to themselves. Alternative conventions exist.

Solutions:

3 Graphs



(a) Represent the above graph as an adjacency list and adjacency matrix

A → [B, E, F]

B → [D]

C → []

D → [C, E]

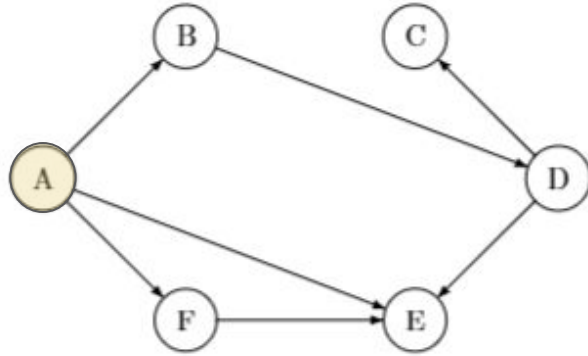
E → []

F → [E]

	A	B	C	D	E	F
A	0	1	0	0	1	1
B	0	0	0	1	0	0
C	0	0	0	0	0	0
D	0	0	1	0	1	0
E	0	0	0	0	0	0
F	0	0	0	0	1	0

Solutions:

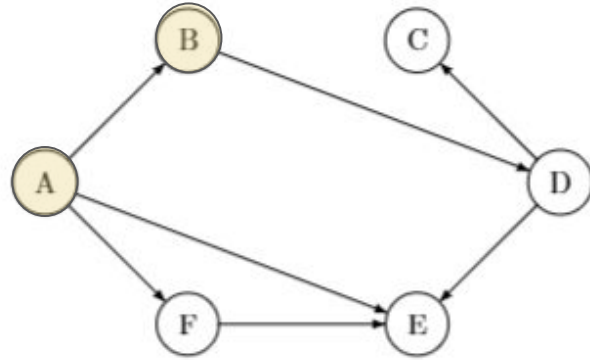
- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order
Start: A

Solutions:

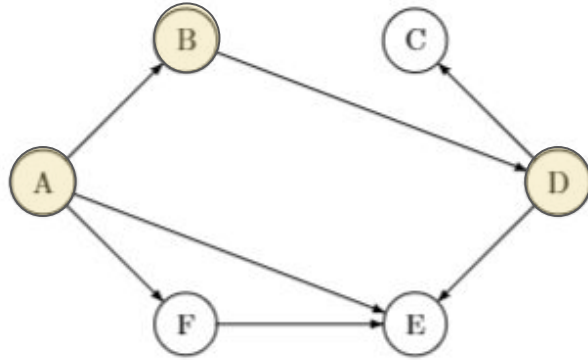
- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order
Start: $A \rightarrow B$

Solutions:

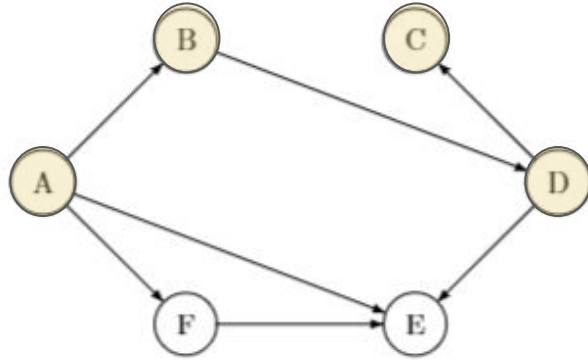
- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order
Start: $A \rightarrow B \rightarrow D$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

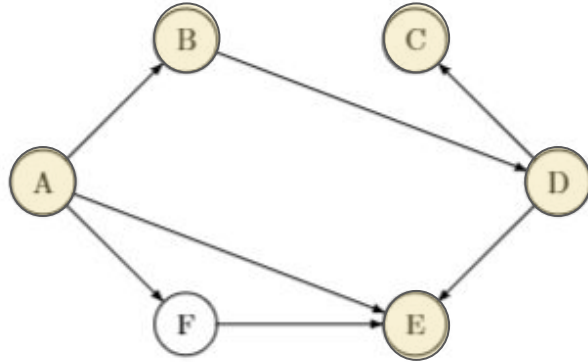


DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

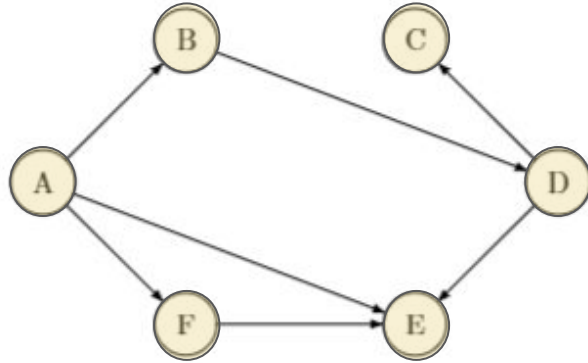


DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.

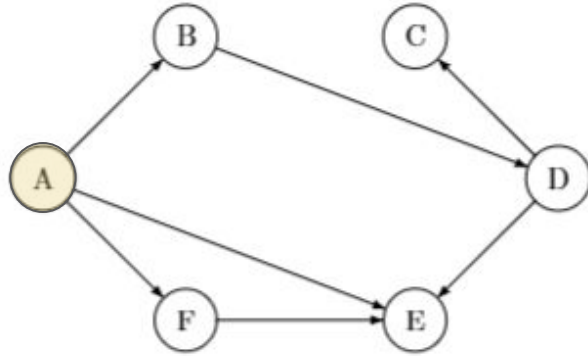


DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



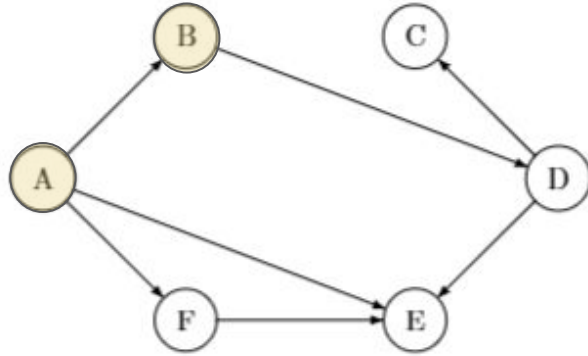
DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



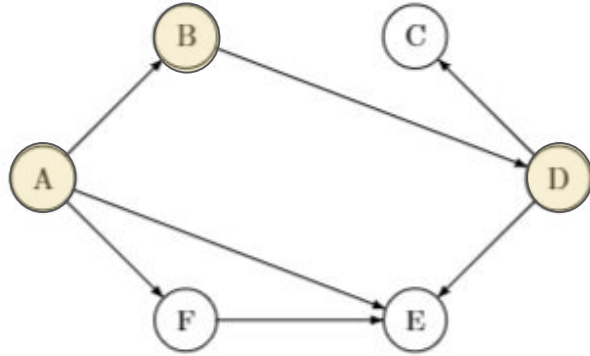
DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



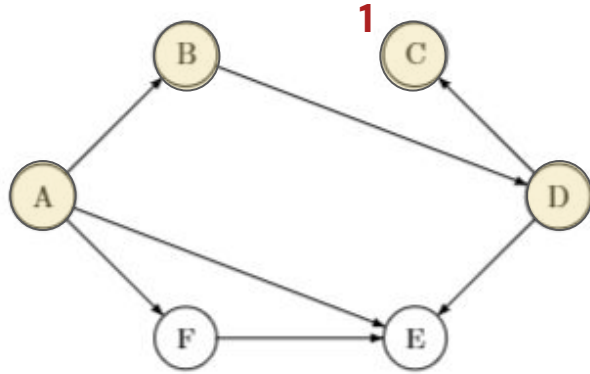
DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

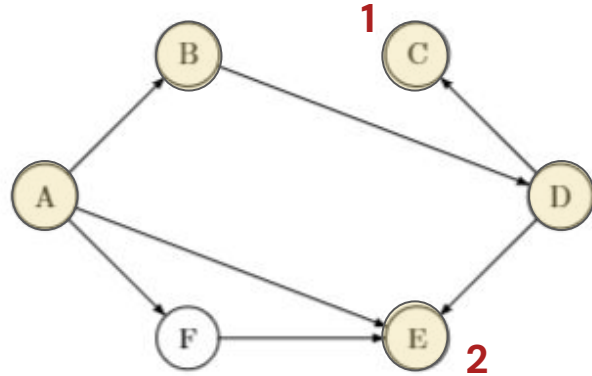
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

C

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

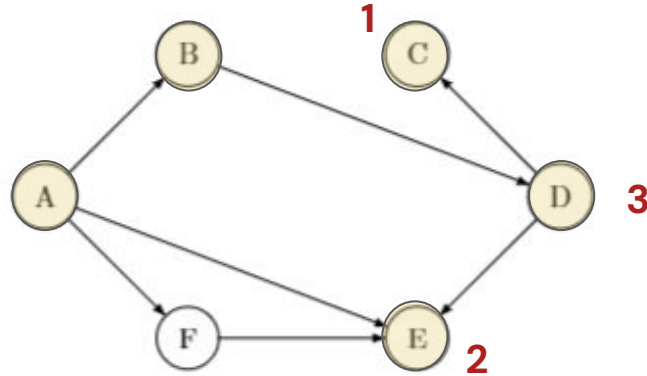
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

$C \rightarrow E$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

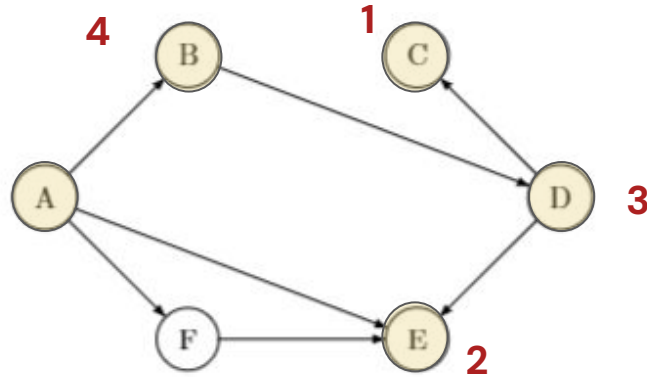
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

$C \rightarrow E \rightarrow D$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

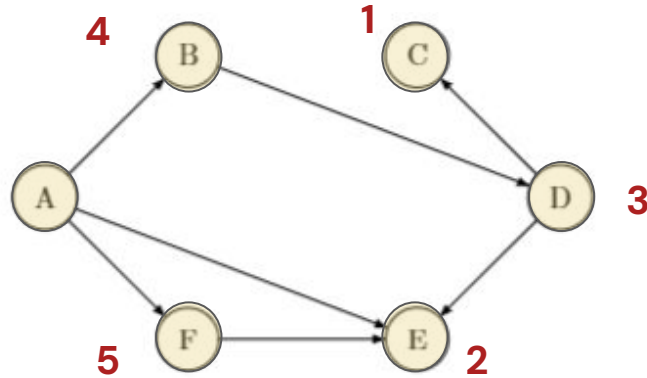
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

$C \rightarrow E \rightarrow D \rightarrow B$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

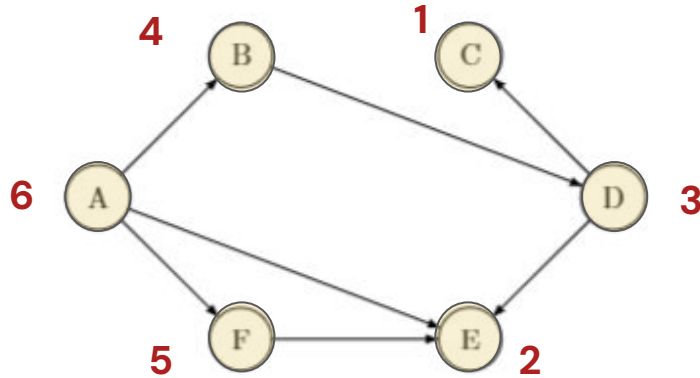
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

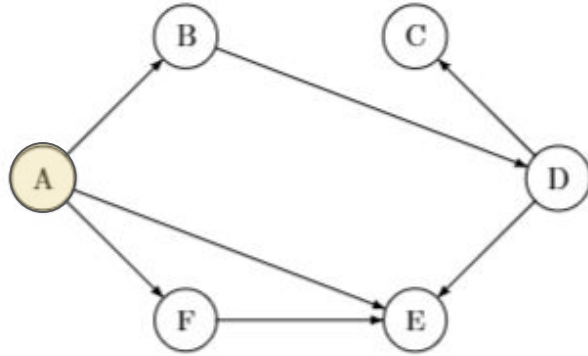
Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F \rightarrow A$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

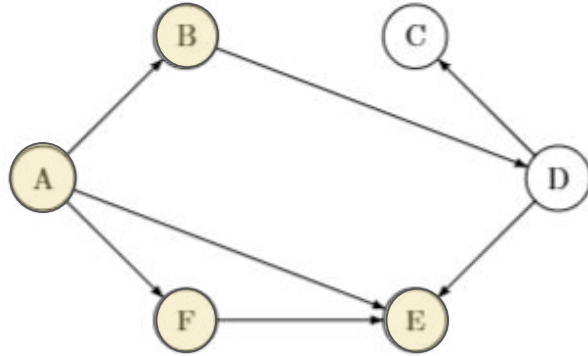
$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F \rightarrow A$

BFS

A

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

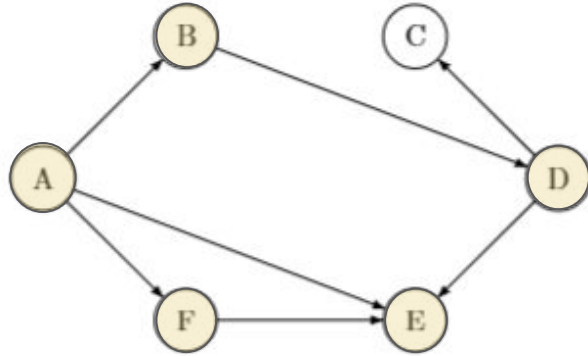
$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F \rightarrow A$

BFS

$A \rightarrow B \rightarrow E \rightarrow F$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

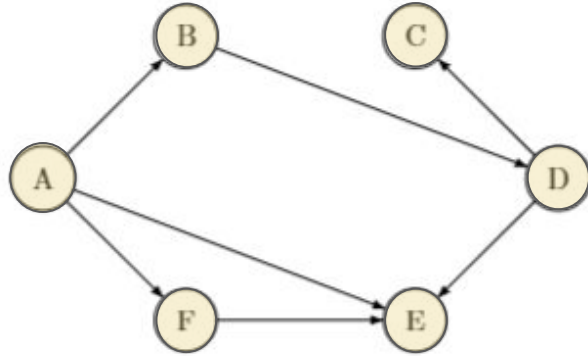
$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F \rightarrow A$

BFS

$A \rightarrow B \rightarrow E \rightarrow F \rightarrow D$

Solutions:

- (b) Run depth first search (DFS) preorder, DFS postorder, and breadth first search (BFS) on the graph above, starting from node A. List the order in which each node is first visited. Whenever there is a choice of which node to visit next, visit nodes in alphabetical order.



DFS pre-order

Start: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$

DFS post-order

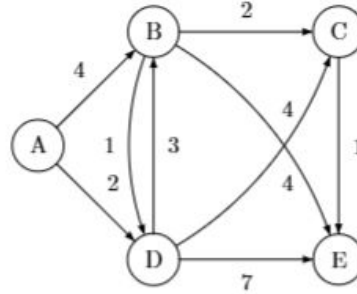
$C \rightarrow E \rightarrow D \rightarrow B \rightarrow F \rightarrow A$

BFS

$A \rightarrow B \rightarrow E \rightarrow F \rightarrow D \rightarrow C$

More Graph Practice!

4 Dijkstra's Algorithm



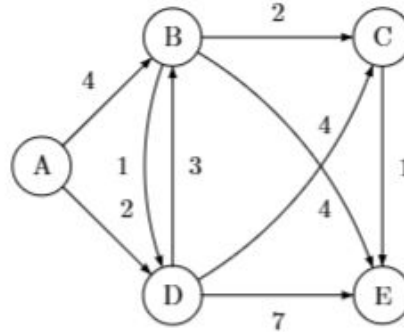
- (a) Given the above graph, run Dijkstra's algorithm starting at node A. For each iteration, write down the entire state of the algorithm. This includes the value $\text{dist}(v)$ for all vertices v as well as what node was popped off of the fringe for that iteration.

Note: If you want to keep track of the vertices traversed along the shortest paths from A to every other node in the graph, you will need to maintain an `edgeTo` array.

v	Init	Pop	Pop	Pop	Pop	Pop
<i>A</i>						
<i>B</i>						
<i>C</i>						
<i>D</i>						
<i>E</i>						

Solution

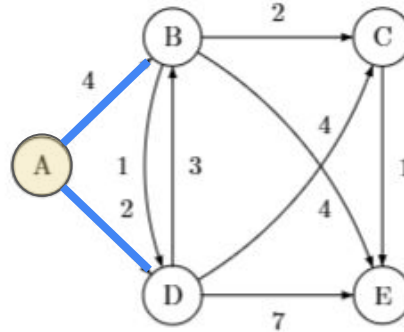
4 Dijkstra's Algorithm



<i>v</i>	Init
<i>A</i>	0
<i>B</i>	∞
<i>C</i>	∞
<i>D</i>	∞
<i>E</i>	∞

Solution

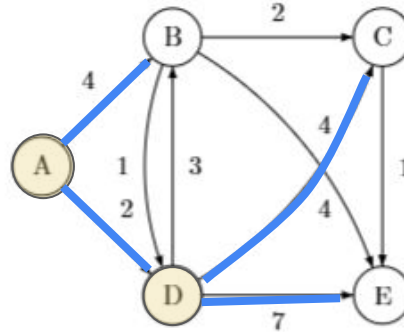
4 Dijkstra's Algorithm



<i>v</i>	Init	Pop A
<i>A</i>	0	0
<i>B</i>	∞	4
<i>C</i>	∞	∞
<i>D</i>	∞	2
<i>E</i>	∞	∞

Solution

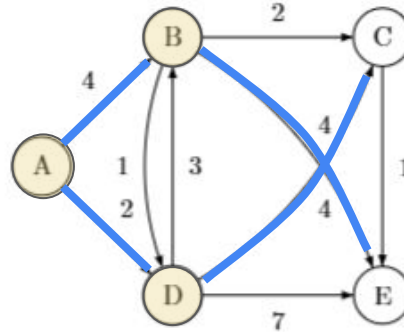
4 Dijkstra's Algorithm



v	Init	Pop A	Pop D
A	0	0	0
B	∞	4	4
C	∞	∞	6
D	∞	2	2
E	∞	∞	9

Solution

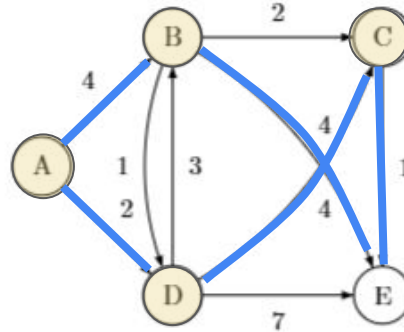
4 Dijkstra's Algorithm



<i>v</i>	Init	Pop A	Pop D	Pop B
<i>A</i>	0	0	0	0
<i>B</i>	∞	4	4	4
<i>C</i>	∞	∞	6	6
<i>D</i>	∞	2	2	2
<i>E</i>	∞	∞	9	8

Solution

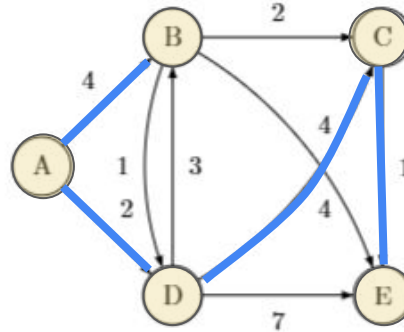
4 Dijkstra's Algorithm



<i>v</i>	Init	Pop A	Pop D	Pop B	Pop C
<i>A</i>	0	0	0	0	0
<i>B</i>	∞	4	4	4	4
<i>C</i>	∞	∞	6	6	6
<i>D</i>	∞	2	2	2	2
<i>E</i>	∞	∞	9	8	7

Solution

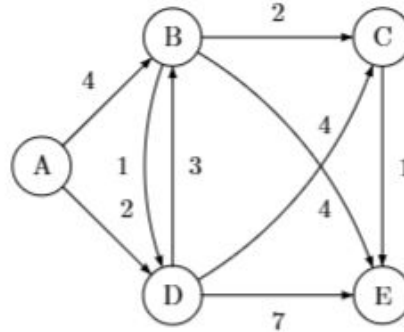
4 Dijkstra's Algorithm



<i>v</i>	Init	Pop A	Pop D	Pop B	Pop C	Pop E
<i>A</i>	0	0	0	0	0	0
<i>B</i>	∞	4	4	4	4	4
<i>C</i>	∞	∞	6	6	6	6
<i>D</i>	∞	2	2	2	2	2
<i>E</i>	∞	∞	9	8	7	7

More Graph Practice!

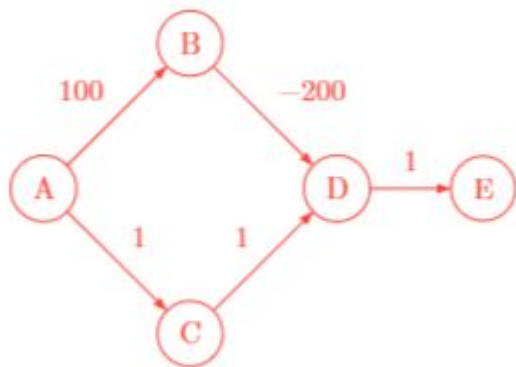
4 Dijkstra's Algorithm



- (b) What must be true about our graph in order to guarantee Dijkstra's will return the shortest path's tree to every vertex? Draw an example of a graph that demonstrates why Dijkstra's might fail if we do not satisfy this condition.

Solution

- (b) What must be true about our graph in order to guarantee Dijkstra's will return the shortest path's tree to every vertex? Draw an example of a graph that demonstrates why Dijkstra's might fail if we do not satisfy this condition.

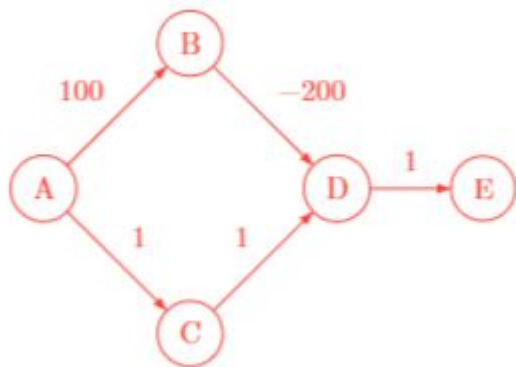


Dijkstra's can fail on graphs with negative edge weights! (not all the time, but enough to wary...)

- If we run Dijkstra's starting at A, it will output: $A \rightarrow C \rightarrow D \rightarrow E$
- This is wrong! The correct shortest path to E is: $A \rightarrow B \rightarrow D \rightarrow E$

Solution

- (b) What must be true about our graph in order to guarantee Dijkstra's will return the shortest path's tree to every vertex? Draw an example of a graph that demonstrates why Dijkstra's might fail if we do not satisfy this condition.



Answer: All edge weights must be positive to *guarantee* Dijkstra's output is correct.

Dijkstra's can fail on graphs with negative edge weights! (not all the time, but enough to wary...)

- If we run Dijkstra's starting at A, it will output: $A \rightarrow C \rightarrow D \rightarrow E$
- This is wrong! The correct shortest path to E is: $A \rightarrow B \rightarrow D \rightarrow E$
- Why does it fail? Dijkstra's pops off the smallest outgoing edge from A, which is $A \rightarrow C$. It assumes that the path can only increase if other paths with non-minimum edges are considered. This does not hold when we have a negative edge weight later on in the path!

Hashing

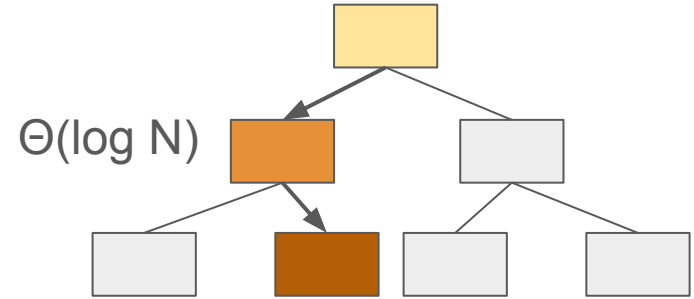
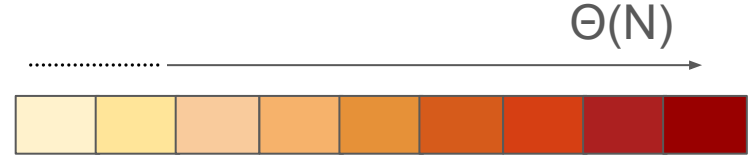
CS 61B MT2 Review Session



Why Hashing? Runtime Analysis!

Data structures we've seen so far...

	contains (x)	add (x)
ArraySet	$\Theta(N)$	$\Theta(N)$
BST	$O(N)$	$O(N)$
2-3 Tree	$\Theta(\log N)$	$\Theta(\log N)$
LLRB	$\Theta(\log N)$	$\Theta(\log N)$



Q: Can we achieve even **better** runtime for both operations?

A: Yes!



Goal: Store a collection of Key-Value pairs.

How to use data as indices?

Hash Code!

Big idea: any object of any type → integer hash value

To stay efficient, keep hash values in a smaller range.

- By wrapping.

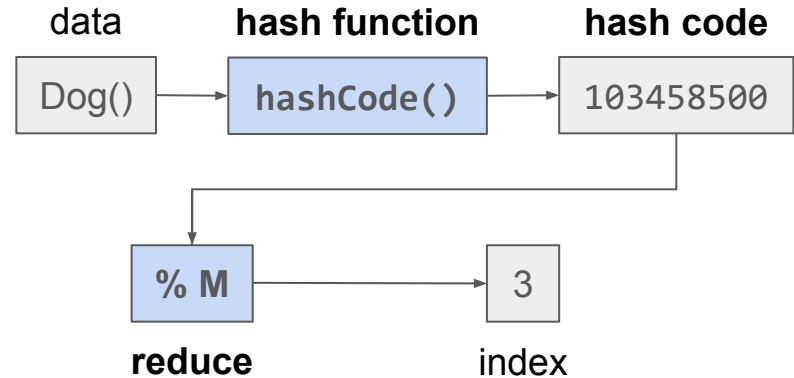
`Math.floorMod(key.hashCode(), backingArr.length)` # floorMod accounts for neg ints, % does not



Hashing Functions + “Reducing”

Consider this case: how can we choose which bucket index to put a Dog Object?

- **Hash function** to assign the Dog Object a **hash code** (int representation of itself)
- **Reduce** this hash code (typically using % and the total # of buckets) to a valid bucket **index**.
 - Math.floorMod for negative hash codes.
- Now we know where to put our Dog Object!



Valid & Good Hash Codes!

Valid (must have)

1. **Deterministic**: The `hashCode()` function of two objects A and B who are equal to each other (`A.equals(B) == true`) have the same hashcode.
 - Hash function only relies on attributes of the object that are reflected in `.equals()`
 - Note: 2 objects with the same hashcode aren't necessarily equal
2. **Consistent**: The `hashCode()` function returns the same integer every time it is called on the same instance of an object
 - hashcode must be independent of time/stopwatches, random number generators, etc.

Good (not must have, but recommended)

1. Valid
2. Spread: distributes elements uniformly across buckets
3. Efficient: relatively quick to compute (ideally $O(1)$)

Writing Hash Functions

Java has a built-in `.hashCode()` hash function that you can call on any Object, but you can also override Java's built-in method and write your own!

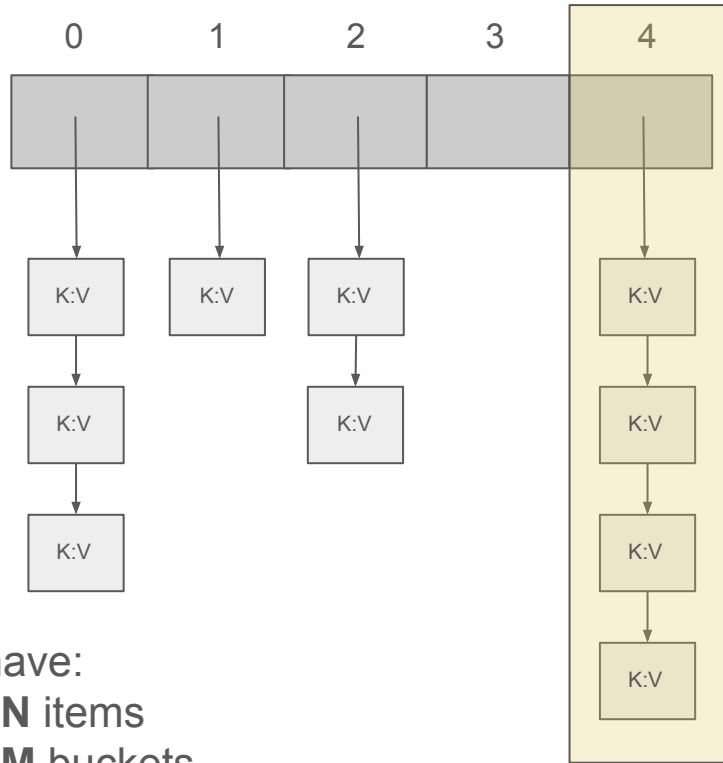
Why write a custom hash function?

- To have consistency between `equals` and `hashCode`
- If you override `.equals()`, you **MUST** override `.hashCode()` too
 - Otherwise, contains breaks and you may add duplicate Objects

Don't mutate an Object being used as a key once you add it to your HashMap!

- `.contains(K key)` may fail because once you change that Object, it may have a new hashcode (this depends on what the attributes hashcode function uses)

Separate Chaining



We have:

- **N** items
- **M** buckets

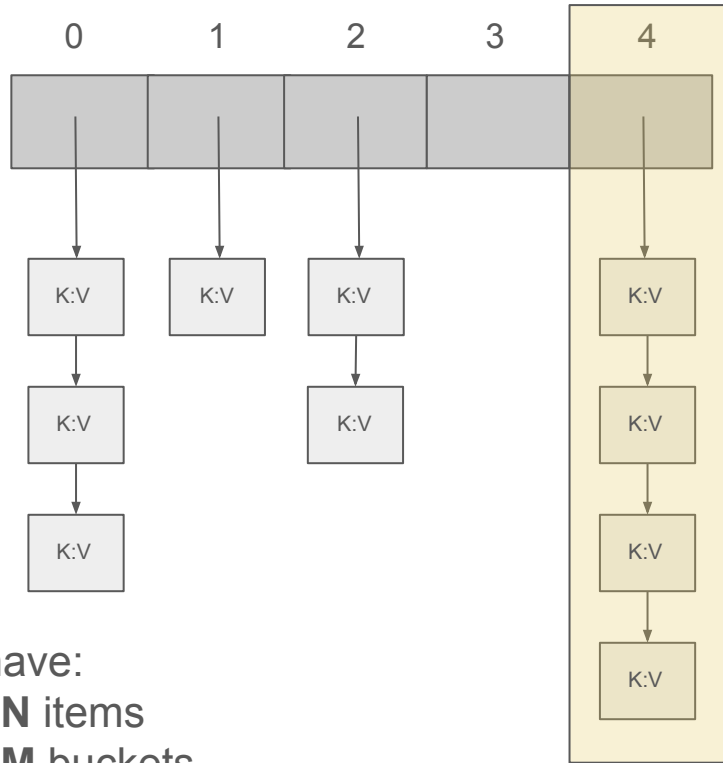
Structure of Hash Table:

- Underlying Array with “buckets”
- Buckets point to a Linked List
 - Elements are placed in the same bucket if their hashcode is the same

“Bucket” at index 4



Separate Chaining



We have:

- **N** items
- **M** buckets

Structure of Hash Table:

- Underlying Array with “buckets”
- Buckets point to a Linked List
 - Elements are placed in the same bucket if their hashcode is the same

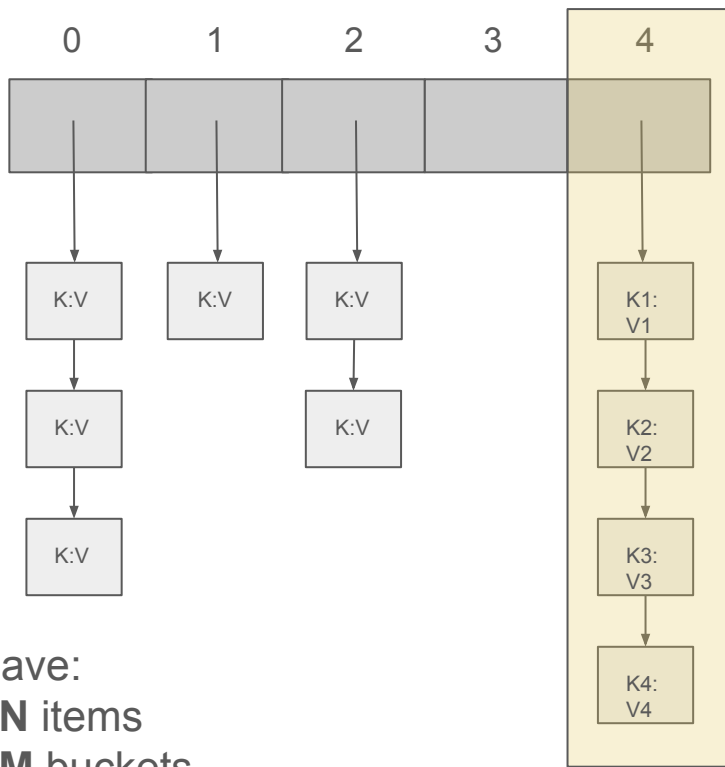
How is this relevant to .contains()?

- Knowing which bucket our target element is in saves us from searching all other buckets.



“Bucket” at index 4

Separate Chaining



We have:

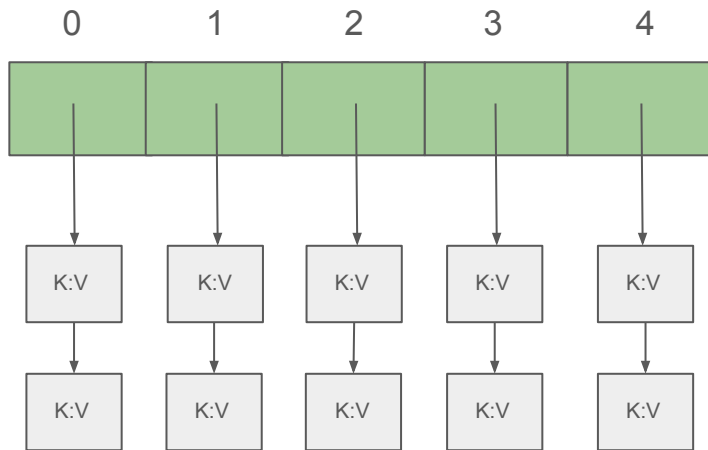
- **N** items
- **M** buckets

How do we know what bucket to search? What bucket to add to?

- We search a hashmap by key, because keys are always unique
- Note: In bucket 4, K1-4 all have hashCode=4, but K1-4 are not necessarily equal to each other

“Bucket” at index 4

Separate Chaining II



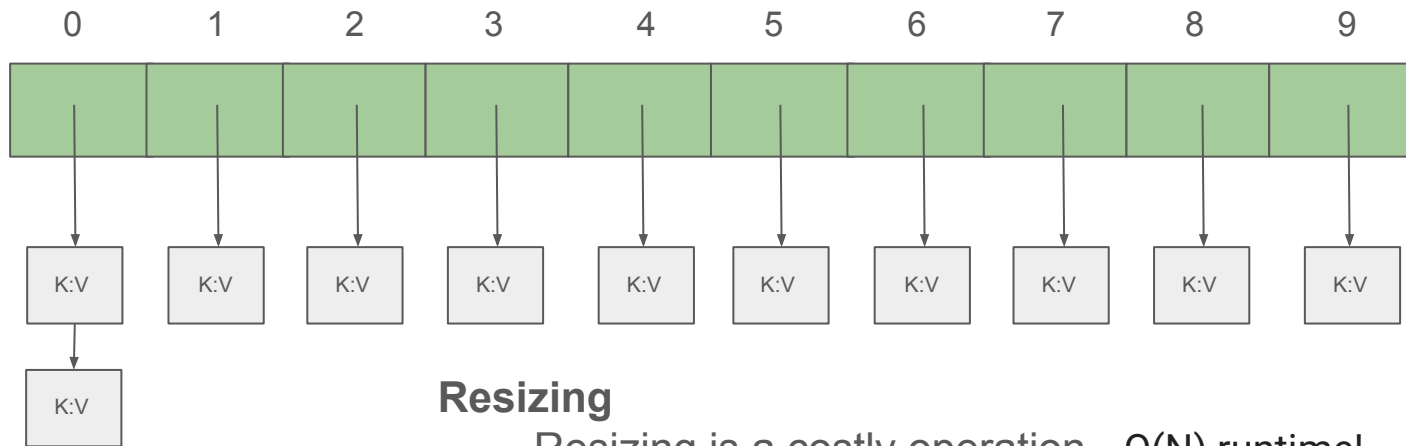
We have:

- **10** items
- **5** buckets

Let's look at a more evenly distributed Hash Table now.

- Less depth (length) of buckets
 - Allows **.contains** runtime to be amortized (average in the long run) to $\Theta(1)$.
- Consider N/M elements per bucket.
 - $10/5 = 2$ elements per bucket!

Separate Chaining II



Resizing

- Resizing is a costly operation - $\Theta(N)$ runtime!
 - However, resizing is **not** common
 - We achieve this by resizing geometrically.
- Amortized runtime for **.add()** is still $\Theta(1)$.

We **now** have:

- **11** items
- **10** buckets

Resizing and a good hashcode yields us $\Theta(1)$ “get” and “put” operations!

More on Resizing

In general, we resize based on this formula:

- **Load factor** = # items (N) / # buckets (M)

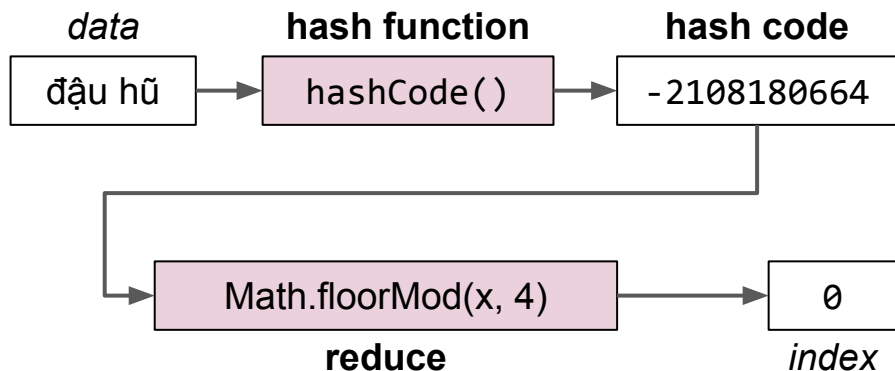
Once this load factor exceeds a certain threshold, we resize geometrically.

Assuming items are evenly distributed, bucket lists will be $\sim N/M$ items long, resulting in $\Theta(N/M)$ runtimes.

- By doubling M every time N gets too big, we ensure that $N/M = O(1)$.
- Thus, worst case runtime for all operations is $\Theta(N/M) = \Theta(1)$
 - unless that operation causes a resize (but will be fine amortized)
 - and assuming even distribution of items.

Summary of Hash Tables

- *Data* is converted into a hash code.
- The **hash code** is then **reduced** to a valid *index*.
- *Data* is then stored in a bucket corresponding to that *index*.
- Resize when load factor N/M exceeds some constant.
- If items are spread out evenly, you get $\Theta(1)$ average runtime.



	contains(x)	add(x)
Bushy BSTs	$\Theta(\log N)$	$\Theta(\log N)$
Separate Chaining Hash Table With No Resizing	$\Theta(N)$	$\Theta(N)$
... With Resizing	$\Theta(1)^\dagger$	$\Theta(1)^*\dagger$

*: Amortized.

†: Assuming items are evenly spread.

Hashtable vs HashSet vs HashMap

What is the distinction between hash set, hash map, and hash table?

- A “**hash table**” is a way of storing information, where you have M buckets that store N items. Each item has a “hashCode” that tells you which of M buckets to put that item in.
- A “**hash set**” is an implementation of the Set ADT using the “hash table” as its backing data structure.
- A “**hash map**” is an implementation of the Map ADT using the “hash table” as its backing data structure.

7 Spread and Validity

For Reference:

```
@Override
public boolean equals(Object o) {
    if (!(o instanceof Dog)) {
        return false;
    }
    Dog other = (Dog) o;
    return this.name.equals(other.name) && this.microchipID == other.microchipID;
}
```


7a INVALID

```
@Override  
    public int hashCode() {  
        return this.age;  
    }
```

Equality / `.equals()` does not depend on age.

Check if hash codes are valid: if two objects are `.equals()`, do those two objects necessarily have the same hash code? In this case, no → invalid.

Also, age is mutable.

7a VALID WITH GOOD SPREAD

```
@Override  
public int hashCode() {  
    return this.microchipID;  
}
```

GOOD SPREAD: Since microchipID can take on a large range of values, it is good.

VALID: microchipID is present in .equals() & immutable

7a VALID WITH BAD SPREAD

```
@Override  
public int hashCode() {  
    return (int) this.name.charAt(0);  
}
```

VALID: name contributes to .equals() & immutable

BAD SPREAD: There is not a large variety of characters in name.charAt(0)

7a INVALID

```
@Override  
public int hashCode() {  
    return this.owner.hashCode();  
}
```

INVALID: 2 dogs that are equal might not have same owner.

- Owner is not present in .equals()

7b

- (b) If a `HashMap` contains two or more items with equal keys, how might have this happened? Give one reason. Assume that we are referring to the built in java `HashMap`. Note that this part is independent from the previous; i.e. the keys aren't necessarily `Dogs`.

7b

- (b) If a HashMap contains two or more items with equal keys, how might have this happened? Give one reason. Assume that we are referring to the built in java HashMap. Note that this part is independent from the previous; i.e. the keys aren't necessarily Dogs.

Two possible reasons:

- The hash function is invalid/non deterministic
- A key was mutated after insertion such that now its hashCode is the same as another key

5 Hashing Practice

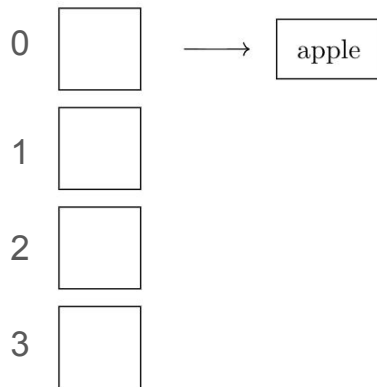
For Reference:

Given the provided `hashCode` implementation, hash the items listed below with separate chaining (the first item is already inserted for you). Assume the load factor is 1. Use geometric resizing with a resize factor of 2.

You may draw more boxes to extend the array when you need to resize.

```
/** Returns 0 if word begins with 'a', 1 if it begins with 'b', etc. */  
public int hashCode() {  
    return (int)(word.charAt(0) - 'a');  
}
```

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`

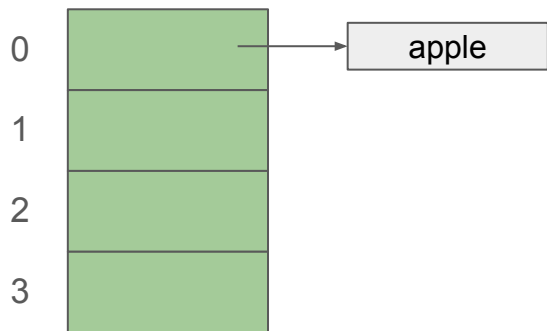


a:0, b:1, c:2, d:3, e:4, f:5, g:6,...

5

Load factor = 1

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`



items = 1

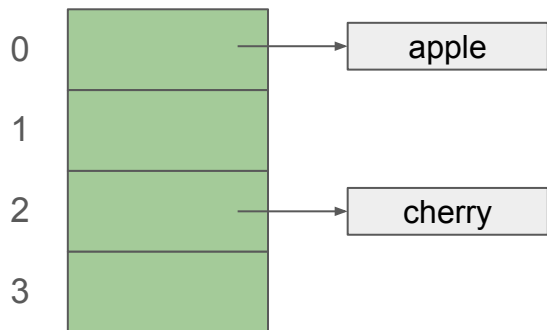
buckets = 4

items / # buckets = 0.25

5

Load factor = 1

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`



items = 2

buckets = 4

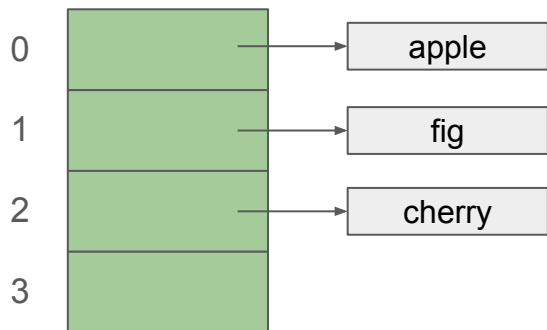
items / # buckets = 0.5

hashCode of "cherry" = 2 % 4 = 2

5

Load factor = 1

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`



items = 3

buckets = 4

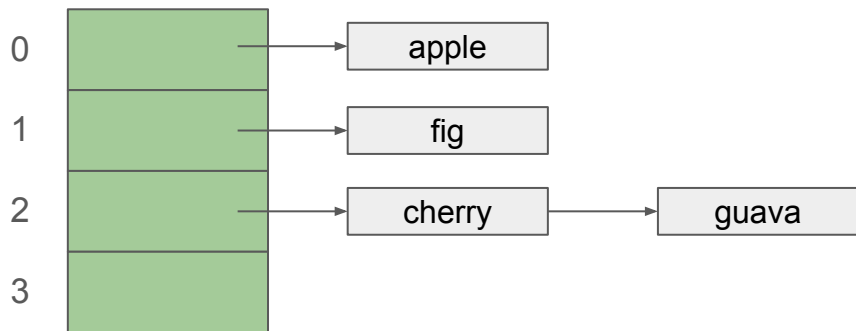
items / # buckets = 0.75

hashCode of "fig" = 5 % 4 = 1

5

Load factor = 1

["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]



items = 4

buckets = 4

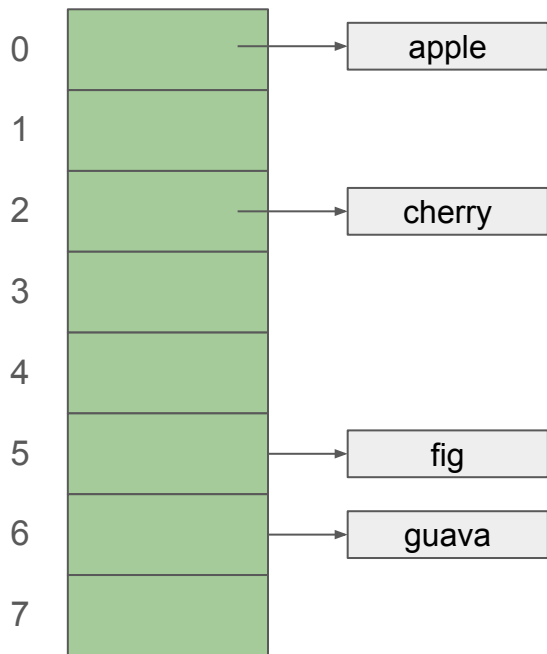
items / # buckets = 1

N / M >= load factor = 1, which is too large. Time to resize hash table and redistribute elements!

hashCode of "guava" = 6 % 4 = 2

5

["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]



items = 4

buckets = 8

items / # buckets = 0.5

hashCode of "apple" = 0 % 8 = 0

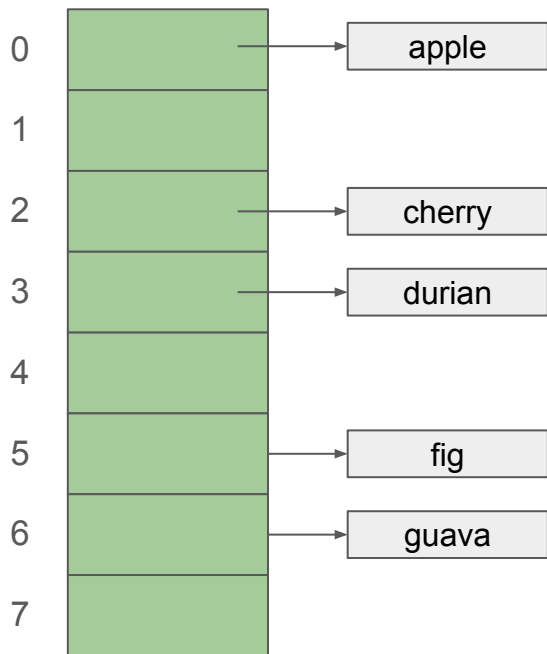
hashCode of "cherry" = 2 % 8 = 2

hashCode of "fig" = 5 % 8 = 5

hashCode of "guava" = 6 % 8 = 6

5

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`



items = 5

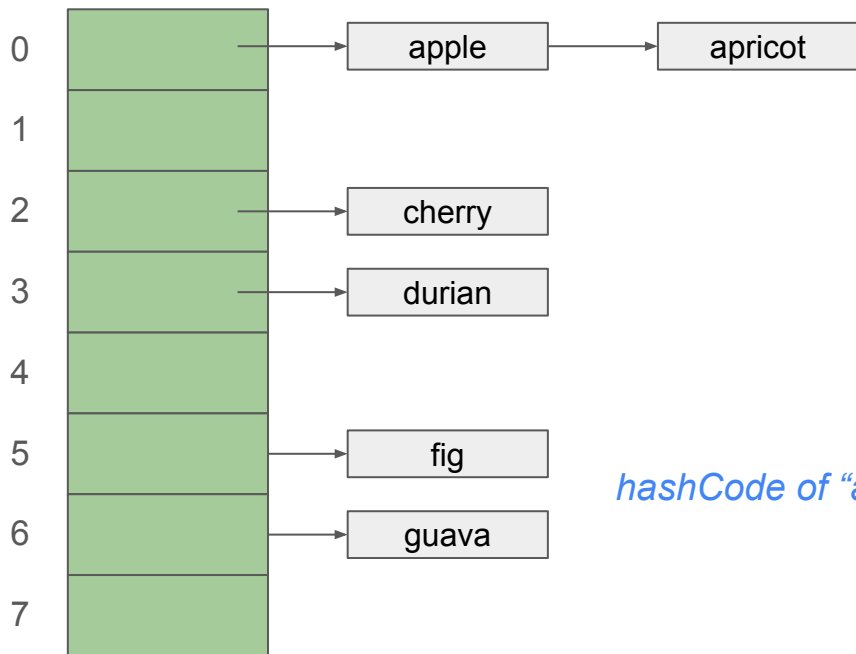
buckets = 8

items / # buckets = 0.625

hashCode of "durian" = 3 % 8 = 3

5

["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]



items = 6

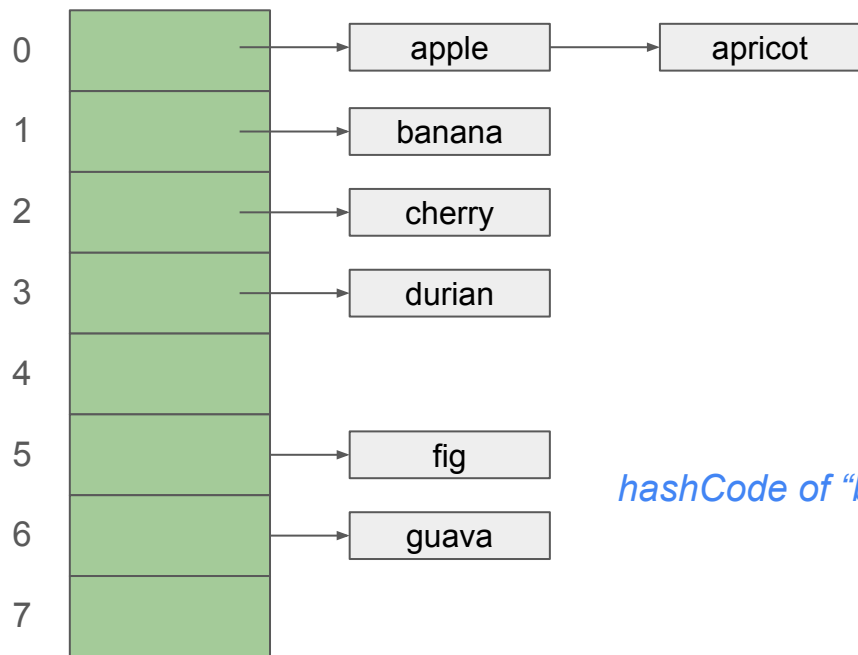
buckets = 8

items / # buckets = 0.75

hashCode of "apricot" = 1 % 8 = 1

5

`["apple", "cherry", "fig", "guava", "durian", "apricot", "banana"]`



items = 7

buckets = 8

items / # buckets = 0.875

hashCode of "banana" = 1 % 8 = 1

6 LLRB-Chaining

6 (SP21 MT2)

Note for this question: Do **not** make any assumptions about the way our Hash Function distributes items

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

(a) Using this hash table,

The runtime of **contains** is, in the worst case:

Runtime: Θ (_____)

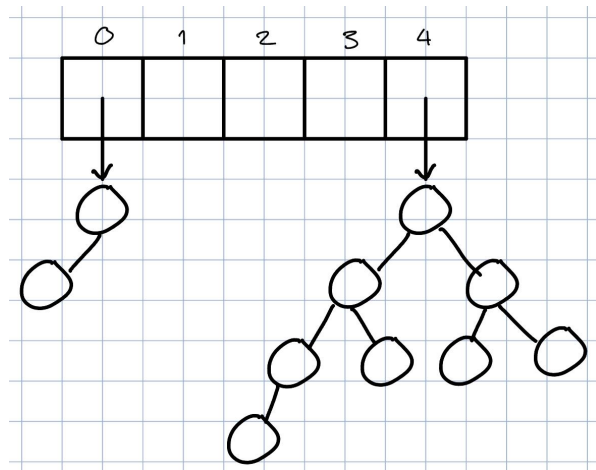
The runtime of **contains** is, in the best case:

Runtime: Θ (_____)

(b) Evaluate the following statements as **true** or **false**, and briefly explain why.

“One advantage of using an LLRB for buckets is that it becomes possible to efficiently iterate over the keys in ascending order”

“Assuming items in the hash table have good spread, we expect that an LLRB bucket would yield significantly better performance for **contains** and **add** than using an **ArrayList**”



6 (SP21 MT2)

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

(a) Using this hash table,

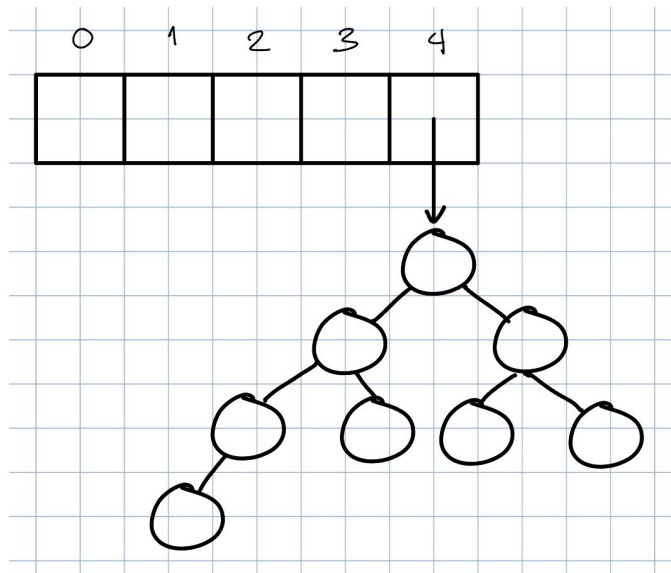
The runtime of **contains** is, in the worst case:

Runtime: $\Theta \left(\frac{\log N}{ } \right)$

The question does **not** make any assumptions about the way our Hash Function distributes items!

In the **worst** case:

- All of our items fall under the same bucket
- Searching this bucket takes **$O(\log N)$ in the worst case.**
 - *From B-Tree Runtimes!*



6 (SP21 MT2)

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

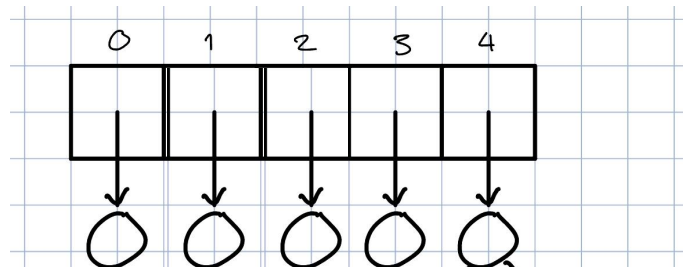
(a) Using this hash table,

The runtime of **contains** is, in the worst case:

Runtime: $\Theta (\text{log } N)$

The runtime of **contains** is, in the best case:

Runtime: $\Theta (1)$



In the **best** case:

- All of our items fall under different buckets
- Only 1 item/bucket = constant time search

6 (SP21 MT2)

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

(b) Evaluate the following statements as **true** or **false**, and briefly explain why.

“One advantage of using an LLRB for buckets is that it becomes possible to efficiently iterate over the keys in ascending order”

FALSE

- Each bucket contains a distinct LLRB
 - Can efficiently iterate through keys in one bucket in ascending order
- Difficult to iterate over all of keys still

6 (SP21 MT2)

Suppose that we have a hash table implemented using separate chaining **with an LLRB in each bucket** instead of a Linked List. Assume that the hash table's key implements **Comparable**.

(b) Evaluate the following statements as **true** or **false**, and briefly explain why.

“Assuming items in the hash table have good spread, we expect that an LLRB bucket would yield significantly better performance for **contains** and **add** than using an **ArrayList**”

FALSE

- Each bucket contains a distinct LLRB
- If good spread, items should be well spread throughout buckets = small buckets
 - N/M items per bucket (constant)
 - Tree buckets would help if buckets were big